



**M. Kumarasamy
College of Engineering**

NAAC Accredited Autonomous Institution

Approved by AICTE & Affiliated to Anna University
ISO 9001:2015 Certified Institution
Thalavapalayam, Karur - 639 113, TAMILNADU.



**AGC1361 & AGILE METHODOLOGY
AGILE NEXUS - A PREDICTIVE FRAMEWORK FOR SPRINT
HEALTH, PRODUCTIVITY, AND DEPENDENCY
MANAGEMENT**

A PROJECT REPORT

Submitted by

PRANEETA R (927623BAD078)

ROOBA B (927623BAD092)

SADHANA G (927623BAD095)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

M.KUMARASAMYCOLLEGE OF ENGINEERING, KARUR

DECEMBER 2025

M. KUMARASAMY COLLEGE OF ENGINEERING, KARUR
(Autonomous Institution affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

This is to certify that this project report on “**AGILE NEXUS: A PREDICTIVE FRAMEWORK FOR SPRINT HEALTH, PRODUCTIVITY, AND DEPENDENCY MANAGEMENT**” is the bonafide work of **PRANEETA R (927623BAD078), ROOBA B (927623BAD092) & SADHANA G (927623BAD095)** who carried out the project work during the academic year 2025 - 2026 under my supervision.

SIGNATURE

Dr. S. BHARANINAYAGI M.E., Ph.D.,
SUPERVISOR,

Department of Artificial Intelligence and

Machine Learning,

M. Kumarasamy College of Engineering,

Thalavapalayam, Karur - 639 113.

SIGNATURE

Dr. A. SELVI, M.Tech., Ph.D.,

ASSOCIATE PROFESSOR,

HEAD OF THE DEPARTMENT,

Department of Artificial Intelligence,

M. Kumarasamy College of Engineering,

Thalavapalayam, Karur - 639 113.

This Project Work (AGC1361) report has been submitted for the V Semester Project viva voce Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

M. KUMARASAMY COLLEGE OF ENGINEERING
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

VISION OF THE DEPARTMENT

To excel in education, innovation, and research in Artificial Intelligence and Data Science to fulfil industrial demands and societal expectations.

MISSION OF THE DEPARTMENT

M1: To educate future engineers with solid fundamentals, continually improving teaching methods using modern tools.

M2: To collaborate with industry and offer top-notch facilities in a conducive learning environment.

M3: To foster skilled engineers and ethical innovation in AI and Data Science for global recognition and impactful research.

M4: To tackle the societal challenge of producing capable professionals by instilling employability skills and human values.

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

Graduates will be able to:

PEO 1: Compete on a global scale for a professional career in Artificial Intelligence and Data Science.

PEO2: Provide industry-specific solutions for the society with effective communication and ethics.

PEO3: Hone their professional skills through research and lifelong learning initiatives.

Mapping of Programme Educational Objectives with Mission of Department

PEOs/Department Mission Statements	M1	M2	M3	M4
PEO1	3	3	2	3
PEO2	3	3	2	2
PEO3	3	2	3	2

1: Slight (Low)

2: Moderate (Medium)

3: Substantial (High)

PROGRAMME OUTCOMES (POs)

PO1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with engineering community such as, being able to comprehend and write effective reports and design documentation, make effective presentations, give and receive clear instructions.

PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological.

PROGRAMME SPECIFIC OUTCOMES (PSOs)

PSO1: Capable of finding the important factors in large datasets, simplify the data, and improve predictive model accuracy. change

PSO2: Capable of analyzing and providing a solution to a given real-world problem by designing an effective program.

Mapping of Programme Educational Objectives with Programme Outcomes and Programme Specific Outcomes:

PEOs/ POs & PSOs	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO 11	PO 12	PSO 1	PSO 2
PEO1	3	2	2	2	3	2	3	3	1	2	3	1	3	1
PEO2	2	2	3	2	3	3	3	2	2	3	2	3	3	2
PEO3	3	3	2	3	3	2	3	3	3	2	3	3	2	3

1: Slight (Low)

2: Moderate (Medium)

3: Substantial (High)

Abstract	POs Mapped	PSOs Mapped
The Agile Nexus is an intelligent, data-driven framework designed to enhance agile software development by addressing sprint failures, productivity fluctuations, and dependency bottlenecks. The system unifies data engineering, analytics, and machine learning to continuously monitor, predict, and optimize team performance. It includes three core modules: (1) a Sprint Risk Prediction Model using Random Forest and XGBoost to evaluate sprint health based on meeting hours, calendar density, and focus time; (2) a Productivity Heatmap Analyzer using K-Means and SVR to identify peak working hours and generate scheduling insights; and (3) an Intelligent Dependency Tracker that detects cascading delays across interlinked sprint tasks. By integrating ETL pipelines with data from calendars, communication tools, and code repositories, the system transforms raw activity logs into structured metrics through preprocessing and feature engineering. Real-time dashboards offer visual insights into risks, productivity patterns, and workload trends, enabling informed decision-making. Through continuous learning and automated alerts, Agile Nexus improves sprint predictability, reduces bottlenecks, and supports sustained team efficiency.	PO1(3) PO2(3) PO3(3) PO4(2) PO5(3) PO6(2) PO7(1) PO8(2) PO9(3) PO10(2) PO11(3) PO12(2)	PSO1(3) PSO2(3)

1- LOW, 2-MEDIUM, 3-HIGH

SDG Goal		Remarks
SDG 9	This project supports Sustainable Development Goal 9 by promoting technological innovation, strengthening digital infrastructure, and enabling advanced AI-driven solutions for workflow optimization.	Enhances industry efficiency, boosts innovation in software development, and supports sustainable digital transformation.

ABSTRACT

The Agile Nexus is an intelligent, data-driven framework developed to enhance agile software development performance by addressing recurring challenges such as sprint failures, productivity variations, and cross-team dependency delays. The system integrates multiple analytics and machine learning components into a unified architecture that continuously monitors, predicts, and optimizes team efficiency. It comprises three core modules: (1) a Sprint Risk Prediction Model that leverages Random Forest and XGBoost algorithms to forecast sprint health based on parameters like meeting hours, calendar density, and focus time; (2) a Productivity Heatmap Analyzer, employing K-Means clustering and Support Vector Regression (SVR) to identify peak productivity hours and generate actionable scheduling insights; and (3) an Intelligent Dependency Tracker that maps interlinked sprint tasks to detect potential cascade failures and alert teams proactively. By combining ETL data integration from diverse organizational sources such as calendars, communication tools, and commit histories, Agile Nexus transforms raw operational data into meaningful metrics through preprocessing, normalization, and feature engineering stages. The system provides real-time visual dashboards for sprint monitoring, risk visualization, and productivity mapping, thereby enabling better decision-making and workload management. Through continuous learning and performance feedback loops, Agile Nexus empowers agile teams to predict sprint outcomes, minimize bottlenecks, and achieve sustained improvement in overall delivery performance and project predictability.

KEYWORDS: *Agile Nexus, sprint risk prediction, Random Forest, XGBoost, productivity heatmap, K-Means clustering, Support Vector Regression (SVR), intelligent dependency tracking, cascade failure detection, ETL data integration, feature engineering, predictive analytics, agile performance optimization, workflow bottleneck detection, real-time sprint monitoring*

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO.
	ABSTRACT	vii
	LIST OF TABLES	x
	LIST OF FIGURES	xi
	LIST OF ABBREVIATIONS	xii
1	INTRODUCTION	1
	1.1 OVERVIEW	2
	1.2 DOMAIN INTRODUCTION	2
	1.2.1 ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING	2
	1.2.2 MACHINE LEARNING: FUNCTIONAL OVERVIEW	2
	1.2.3 FACTORS DRIVING MACHINE LEARNING ADOPTION	3
	1.2.4 TYPES OF MACHINE LEARNING	3
	1.3 PROBLEM STATEMENT	5
	1.4 OBJECTIVE	6
2	LITERATURE REVIEW	7
3	FEASIBILITY STUDY	11
	3.1 FEASIBILITY STUDY	12
	3.1.1 TECHNICAL FEASIBILITY	12
	3.1.2 ECONOMICAL FEASIBILITY	13
	3.1.3 OPERATIONAL FEASIBILITY	13
4	PROJECT METHODOLOGY	14
	4.1 RANDOM FOREST CLASSIFIER	15
	4.2 XGBOOST	16
	4.3 K-MEANS CLUSTER	17
	4.4 SUPPORT VECTOR REGRESSION	17
	4.5 GRAPH NEURAL NETWORKS	18
	4.6 MODULES	20

4.7	MODULE DESCRIPTION	20
4.7.1	DATABASE SETUP	20
4.7.2	ETL PIPELINE	20
4.7.3	DATAPROCESSING AND FEATURE ENGINEERING	21
4.7.4	MACHINE LEARNING MODELS	21
4.7.5	REAL DATA INTEGRATION	22
4.7.6	REAL TIME DASHBOARD	23
4.7.7	NOTIFICATION ENGINE	23
5	RESULT	24
6	CONCLUSION & FUTURE ENHANCEMENT	35
6.1	CONCLUSION	36
6.2	FUTURE ENHANCEMENT	36
	REFERENCES	37
	APPENDIX	40

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
2.1	AI TECHNIQUES FOR AGILE PROJECT INTELLIGENCE AND SPRINT PREDICTION	8
2.2	CLASSIFICATION MODELS USED FOR AGILE PROJECT PREDICTION	9
2.3	AI MODELS FOR ROBUST AGILE PROJECT ANALYSIS AND PREDICTION	10

LIST OF FIGURES

S.NO.	TITLE	PAGE NO
4.1	ARCHITECHTURE DIAGRAM	19
5.1	REGISTRATION & LOGIN PAGE	25
5.2	MANAGER DASHBOARD OVERVIEW	25
5.3	CREATE NEW PROJECT	26
5.4	ADD TEAM MEMBER TO THE PROJECT	26
5.5	AVERA GE TEAM SPRINT RISK SCORE IN MANAGER TEAM DASHBOARD	27
5.6	AVERAGE T EAM PRODUCTIVITY SCORE IN MANAGER TEAM DASHBOARD	27
5.7	TEAM DEPENDENCIES IN MANAGER DASHBOARD	28
5.8	HIGH RISK USERS IN MANAGER DASHBOARD	28
5.9	TEAM RISK GAUGE IN MANAGER DASHBOARD	29
5.10	TEAM RISK ANALYSIS IN MANAGER DASHBOARD	29
5.11	TEAM PRODUCTIVITY HEATMAP IN MANAGER DASHBOARD	30
5.12	REMOVING THE TEAM MEMBERS AND TASK ASSIGNMENT TRACKING IN MANAGER DASHBOARD	30
5.13	ASSIGNING A TASK WITH AI RECOMMENDATIONS	31
5.14	TRACKING THE TASKS	31
5.15	INDIVIDUALS FILTERING DATA IN MANAGER DASHBOARD	32
5.16	SPECIFIC TEAM MEMBER DATA WITH TASKS	32
5.17	TEAM MEMBER DASHBOARD OVERVIEW	33
5.18	TASKS LIST IN MEMBER DASHBOARD	33
5.19	SUBMITTING THE TASK AND NOTIFY THE MANAGER	34

LIST OF ABBREVIATIONS

ACRONYMS

EXPANSIONS

AI	Artificial Intelligence
ML	Machine Learning
ETL	Extract, Transform, Load
CNN	Convolutional Neural Network
RF	Random Forest
XGBoost	Extreme Gradient Boosting
SVR	Support Vector Regression
GNN	Graph Neural Network
K-Means	K-Means Clustering Algorithm
API	Application Programming Interface
DBMS	Database Management System
UI	User Interface

CHAPTER - 1

INTRODUCTION

1.1 OVERVIEW

Agile Nexus is an intelligent, data-driven framework designed to enhance agile software development by predicting sprint risks, optimizing team productivity, and identifying cross-sprint dependencies using advanced analytics and machine learning. The system integrates data from calendars, communication platforms, code repositories, and agile tools through a unified ETL pipeline that prepares meaningful features for analysis. Using models such as Random Forest, XGBoost, K-Means, and SVR, it generates insights through productivity heatmaps, dependency maps, and automated scheduling recommendations. Supported by a robust architecture with multi- database storage, REST APIs, and interactive dashboards, Agile Nexus transforms traditional reactive project management into a proactive, insight-driven process that improves team coordination, workload balance, and overall development efficiency.

1.2 DOMAIN INTRODUCTION

1.2.1 Artificial Intelligence & Machine Learning

Artificial Intelligence (AI) is the field that builds systems capable of performing tasks that normally require human intelligence, while Machine Learning (ML) is a subset of AI that enables systems to learn patterns from data and improve performance without explicit programming. ML models ingest historical records - such as meeting logs, commit histories, and task updates - to discover relationships and make predictions. In Agile Nexus, ML is used to automatically identify risk factors, forecast sprint outcomes, and surface productivity patterns across teams. By converting raw activity streams into meaningful features (e.g., meeting density, focus hours, commit frequency), ML enables proactive decision-making and reduces manual monitoring. It scales to high-dimensional, noisy data and adapts as team behaviour changes, making it ideal for dynamic agile environments. Overall, ML brings automation, timely insights, and data-driven recommendations that help teams prevent delays and optimize delivery.

1.2.2 Machine Learning: Functional Overview

Machine Learning works by training models on historical data so they can learn patterns and make predictions on new unseen data. ML systems consist of multiple processing layers that extract relationships from input features such as meeting hours, commit activity, message counts, and productivity trends. During training, the model adjusts internal parameters (weights) to decrease prediction error. A higher weight indicates stronger influence of that feature on the final result

The data is cleaned, scaled, and transformed into mathematical representations. Algorithms like Random Forest, XGBoost, and Support Vector Regression evaluate these features using complex mathematical operations. Each prediction is made by combining weighted feature values and comparing them to learned patterns from previous sprints. Deep learning-based models further automate feature learning using neural networks, but for agile project prediction, ensemble ML models (Random Forest + XGBoost) are effective and highly explainable. While traditional ML needs manual feature selection, modern AI systems automatically.

1.2.3 Factors Driving Machine Learning Adoption

Machine Learning is widely adopted because it eliminates the need for manual rule creation and complex feature engineering. Earlier, traditional methods like Linear Regression, SVM, and Decision Trees required manual selection of important variables and domain knowledge. These models were not capable of handling large, dynamic, and multi-dimensional data such as agile sprint metrics. ML—especially ensemble techniques like Random Forest and XGBoost—automatically identifies the most influential features, such as meeting density, commit frequency, or communication overload. The feature extraction and optimization are performed internally by the algorithm, reducing human effort and improving accuracy. ML models scale efficiently, adapt to new data, and provide insights that help teams optimize productivity, detect sprint failures early, and manage dependencies more effectively. Thus, ML has gained popularity due to its ability to learn from complex datasets, reduce manual intervention, and offer high predictive capability - making it ideal for Agile Nexus.

1.2.4 Types Of Machine Learning

- **Supervised Learning:**

Random Forest, XGBoost, SVR, Decision Trees

- **Unsupervised Learning:**

K-Means, Hierarchical Clustering

- **Graph-Based Learning:**

Graph Neural Networks (GNNs)

- **Ensemble Techniques:**

Bagging + Boosting mode

Supervised Learning

Supervised learning is a type of machine learning where the model is trained using labeled data — meaning the input and the correct output are already known. The algorithm learns to map inputs to outputs and can then predict results for new, unseen data. It is commonly used for classification and regression tasks.

I. Random Forest

Random Forest is an ensemble model that builds multiple decision trees and combines their outputs to improve accuracy and reduce overfitting. It works well for both classification and regression tasks.

II. Xgboost

XGBoost (Extreme Gradient Boosting) is a high-performance boosting algorithm that builds trees sequentially, with each new tree correcting the errors of the previous ones. It is known for its speed, accuracy, and ability to handle complex datasets.

III. Support Vector Regression (SVR)

SVR uses the principles of Support Vector Machines to perform regression. It fits the best possible curve within a certain margin of error, allowing it to model non-linear relationships effectively.

IV. Decision Trees

A Decision Tree splits the data into branches based on conditions and rules to reach a final prediction. It is easy to interpret and used for both classification and regression tasks.

Unsupervised Learning

Unsupervised learning is used when the data does not have labels. The model tries to learn patterns, structures, or relationships within the data without predefined outputs. It is mainly used for clustering and dimensionality reduction.

I. K-Means Clustering

K-Means divides data into K clusters based on similarity. It works by assigning data points to the nearest cluster center and updating centers until the groups stabilize.

Graph-Based Learning

Graph-based learning is used for data represented as graphs — where nodes represent entities and edges represent relationships. These models capture connections, dependencies, and network structures.

I. Graph Neural Networks (GNNs)

GNNs are deep learning models designed to learn from graph-structured data. They capture relationships between nodes and edges, making them useful for modeling dependencies, networks, and connected systems.

Ensemble Techniques

Ensemble learning combines multiple machine learning models to improve performance, accuracy, and stability. By aggregating multiple weak or strong learners, ensemble models achieve better generalization.

13 PROBLEM STATEMENT

Agile software development relies on continuous planning, rapid iteration, and close collaboration, yet many teams still face recurring challenges that hinder effective sprint execution. Sprint failures often occur due to unexpected delays, unbalanced workloads, and fluctuating productivity patterns that managers cannot detect early. Existing agile tools, such as Jira or Trello, operate primarily as task-tracking systems and depend heavily on manual updates, offering limited real-time intelligence or predictive insights. As a result, critical indicators—such as meeting overload, communication bottlenecks, code activity slowdowns, and dependency conflicts across sprints—remain hidden until they cause significant disruptions. Furthermore, organizational data needed to understand team behaviour is scattered across multiple platforms, including calendars, communication channels, version control systems, and agile boards, making it difficult to generate a unified understanding of sprint health. The absence of an integrated analytical framework prevents teams from identifying risk patterns, forecasting delays, and optimizing productivity based on actual behavioural trends. These gaps lead to last-minute firefighting, inefficient resource allocation, and inconsistent sprint outcomes. Therefore, there is a need for an intelligent, data-driven solution that can consolidate multi-source organizational data, analyze team patterns in real time, and empower agile teams with proactive insights to improve planning reliability, productivity, and delivery efficiency.

14 OBJECTIVES

The primary objective of Agile Nexus is to create an intelligent, data-driven framework that enhances agile project management by providing real-time insights, proactive risk detection, and optimized team coordination. The system aims to integrate diverse organizational data—such as meetings, communication patterns, code activity, and task updates—into a unified analytical platform that supports informed decision-making throughout the sprint cycle. Specific objectives include:

- To consolidate multi-source organizational data from calendars, communication tools, code repositories, and agile boards into a unified ETL pipeline for continuous analysis.
- To identify early indicators of sprint risks by analyzing behavioural, communication, workload, and activity patterns that influence sprint progress.
- To optimize team productivity through data-driven insights such as productivity heatmaps, focus hour identification, and workload distribution analysis.
- To detect cross-team and cross-sprint dependencies that may cause workflow delays or cascading risks, and to visualize their impact on current and future sprints.
- To support proactive decision-making by generating automated scheduling recommendations, workload balancing suggestions, and actionable insights.
- To provide an interactive dashboard and API-driven architecture that enables managers and teams to monitor sprint health, productivity trends, and dependency status in real time.
- To improve overall sprint reliability and delivery efficiency by transforming traditional reactive agile practices into predictive, insight-driven workflows.

CHAPTER-2

LITERATURE REVIEW

2.1 TITLE: Data-Driven Predictive Modelling of Agile Projects Using Explainable Artificial Intelligence

AUTHORS: Ali Akbar Et Al

This paper introduces an explainable machine learning framework for predicting sprint risks using Random Forest, XGBoost, and SHAP. It integrates multi-source agile data such as communication logs, workload metrics, and historical sprint performance. The system identifies critical predictors responsible for sprint failures, improving transparency. SHAP-based explanations help managers understand why a sprint is at risk. The model improves early warning capability and supports proactive planning. It demonstrates how AI enhances predictability and decision-making in agile teams. This serves as the foundational inspiration for Agile Nexus. It processes agile metrics like workload, communication intensity, and sprint timelines. The explainability component identifies key factors behind sprint failures. The study improves trust and decision-making in agile management.

2.2 TITLE: Machine Learning–Driven Sprint Outcome Prediction in Agile Software Teams

AUTHORS: Nair, K.

This study leverages Gradient Boosting and Decision Tree models to predict whether a sprint will succeed or face delays using historic agile performance data. It evaluates patterns in sprint velocity, task completion timelines, and developer workload fluctuations to identify factors that commonly lead to schedule slippage. The model highlights behavioral signals—such as inconsistent task flow and uneven workload distribution—that strongly correlate with late sprint delivery. By producing early and accurate predictions, the system allows teams to adjust resource allocation, redistribute tasks, or re-estimate work before issues escalate. It also emphasizes the importance of analyzing long-term sprint patterns rather than relying solely on short-term observations. Overall, the study strengthens forecasting capabilities, enhances planning transparency, and improves sprint reliability.

2.3. TITLE: Predictive Analytics for Software Project Management Using Ensemble Learning

AUTHORS: Lee, J., & Park, D.

This paper examines the impact of ensemble learning techniques—particularly bagging and boosting—on improving project effort prediction and schedule reliability. It processes resource allocation metrics, dependency data, and past effort logs to build robust risk

models that outperform traditional single-model approaches. Ensemble learning effectively reduces noise in dynamic and unpredictable project environments, leading to more stable and accurate predictions. The study demonstrates how early detection of high-risk tasks allows managers to adjust project plans proactively, preventing cascading delays. It also emphasizes the advantage of integrating multiple weak learners to capture nonlinear patterns in team behavior and resource usage.

2.4. TITLE: Modeling Team Productivity Using Communication and Development Activity Logs

AUTHORS: Rodriguez, M., & Fernandes, L.

This study employs K-Means clustering and SVM to analyze developer communication behavior, coding frequency, and meeting activities to uncover productivity trends across teams. It groups developers into clusters based on their work patterns, highlighting differences such as high-coders, frequent communicators, or low-engagement contributors. The model reveals how excessive meetings, reduced commit activity, or irregular communication correlate with productivity drops. These insights help managers diagnose inefficiencies, optimize team workflows, and balance responsibilities more effectively. The research also proves the importance of behavioral data—beyond simple time tracking—in understanding team performance. By providing actionable analytics, the model supports improved collaboration, increased output efficiency, and better alignment of team resources with project goals.

2.5. TITLE: Hybrid AI Framework for Identifying Risk Patterns in Distributed Agile Team

AUTHORS: Khan, S., & Ibrahim, R.

This study introduces a hybrid model combining SVM and Artificial Neural Networks to detect risk patterns in distributed agile teams working across multiple locations and time zones. It evaluates communication delays, uneven task distribution, and cultural or geographic barriers that commonly disrupt coordination. The hybrid approach enhances predictive accuracy by leveraging SVM's classification strengths and ANN's ability to capture nonlinear team behaviors. The system identifies early signs of risk such as reduced interaction frequency, rising workload imbalances, or delayed issue resolution. It supports remote teams by providing insights into collaboration gaps and emerging conflicts that could jeopardize sprint goals. Additionally, the model adapts well to evolving team behaviors, making it suitable for real-world.

TABLE 2.1 AI Techniques for Agile Project Intelligence and Sprint Prediction

S. No	Title of Study	Authors	Algorithms Used	Data Type Used	Application Area
1	Data-Driven Predictive Modelling of Agile Projects Using Explainable Artificial Intelligence	Ali Akbar Forouzeshejad, Farzad Arabikhan, Alexander Gegov, RahelehJafari, Alexandar Ichtev	Random Forest, XGBoost, SHAP (Explainable AI)	Agile sprint metrics, communication logs, timelines, workload data	Agile sprint metrics, communication logs, timelines, workload data
2	Machine Learning–Driven Sprint Outcome Prediction in Agile Software Teams	Nair, K., & Menon, S.	Gradient Boosting, Decision Trees	Historical sprint velocity, task completionlogs	Sprint outcome forecasting and delay detection.
3	Predictive Analytics for Software Project Management Using Ensemble Learning	Lee, J., & Park, D.	Bagging + Boosting (Ensemble Methods)	Effort hours, resource allocation data, dependency metrics	Effort estimation and schedule risk prediction
4	Modeling Team Productivity Using Communication and Development Activity Logs	Rodriguez, M., & Fernandes, L.	K-Means Clustering, Support Vector Machines	K-Means Clustering, Support Vector Machines	Team productivity analysis and behaviour modelling
5	Hybrid AI Framework for Identifying Risk Patterns in Distributed Agile Teams	Khan, S., & Ibrahim, R	SVM + Artificial Neural Networks (Hybrid Model)	Distributedteam communication patterns, task distribution data	Riskdetection in distributed agile project environments

Table 2.1 summarizes AI techniques used in agile project prediction, including ensemble

models, boosting methods, clustering, and neural networks for sprint forecasting and risk analysis. It highlights workload, and commit data to enhance decision-making .

2.6. TITLE: Classification-Based Sprint Failure Prediction in Agile Software Development

AUTHORS: Thomas, R., & Verma, S.

This study uses Logistic Regression and Decision Tree classifiers to predict whether a sprint will be “successful” or “failed” by analyzing backlog size, developer workload, and story- point allocation patterns. The model identifies early indicators such as overloaded backlogs, uneven task distribution, and excessive work commitments that strongly correlate with sprint failure. It helps teams recognize structural issues in sprint planning and adjust workload before execution begins. The classifiers provide interpretable results, allowing managers to understand the specific factors contributing to failure risk. By automating sprint outcome classification, the study enhances decision-making accuracy and strengthens planning discipline.

2.7. TITLE: Machine Learning Classifiers for Predicting Task Completion Reliability

AUTHORS: Banerjee, A., & Cho, J.

This research applies Naïve Bayes and Random Forest classifiers to categorize tasks as *on-time*, *delayed*, or *at-risk* based on commit timestamps, task logs, and developer activity patterns. The models identify reliability factors such as inconsistent commit frequency, stalled updates, or unusual activity gaps that often lead to delays. By monitoring task-level signals in real time, teams can intervene early and prevent schedule slippage. The study demonstrates how behavioral indicators provide stronger predictive power than static task attributes alone. It improves progress tracking and planning accuracy across ongoing sprints. Overall, the model enhances visibility into task reliability and strengthens real-time project monitoring.

2.8. TITLE: Developer Performance Classification Using Multivariate Agile Metrics

AUTHORS: Oliveira, P., & Duarte, M.

This study uses KNN and SVM classification techniques to evaluate developer performance based on a wide range of agile metrics, including communication frequency, coding hours, issue-handling activity, and meeting load. By analyzing these multidimensional indicators, the model categorizes developers into high, medium, and low productivity groups, capturing variations in work pace, collaboration style, and task engagement. It uncovers behavioral patterns

such as excessive time spent in meetings, inconsistent commit activity, or low interaction during crucial development phases, which are strongly linked to performance fluctuations. The insights allow managers to distribute workload more effectively, identify developers who may need support, and reinforce high-performing behavior. Additionally, the study demonstrates that combining communication and development metrics provides a more accurate performance view than traditional time-based assessments. The model supports continuous improvement, balanced team dynamics, and strategic human-resource planning in agile environments.

2.9. TITLE: Deep Learning Classification of Sprint Health Using Temporal Work Patterns

AUTHORS: Hassan, L., & O'Connor, M.

This study applies LSTM (Long Short-Term Memory) networks to classify sprint health by analyzing sequential work patterns, daily progress logs, and developer activity flows. LSTMs excel at modeling long-term dependencies, enabling the system to detect subtle early- warning signs such as reduced task throughput, irregular coding patterns, or abrupt drops in velocity. The deep learning model captures non-linear shifts in team performance that traditional static models fail to identify. It continuously learns from time-stamped work records, offering near real-time insights into sprint stability. The model identifies periods of rising instability, helping teams take timely corrective actions before sprint goals are jeopardized. It improves prediction accuracy for declining sprint performance and supports robust sprint forecasting.

2.10. TITLE: Hybrid Classification Framework for Identifying Bottlenecks in Agile Workflows

AUTHORS: Wang, F., & Petrova, K.

This study proposes a hybrid classification framework combining Gradient Boosting and Artificial Neural Networks to identify bottlenecks and flow disruptions in agile development pipelines. By analyzing task transitions, dependency delays, and workflow interruptions, the model reveals high-impact areas where tasks tend to stagnate or propagate cascading delays. Gradient Boosting captures key decision patterns, while the ANN component learns complex nonlinear behavior within the workflow, resulting in a more comprehensive detection system. The approach highlights bottleneck nodes, slow-moving dependencies, and resource shortages that restrict sprint flow efficiency. It enables managers to pinpoint systemic issues that affect delivery speed and stability.

TABLE 2.2 Classification Models Used for Agile Project Prediction

S. No.	Title of Study	Authors	Classification Model(s) Used	Input Data Type	Application Area
6	Classification-Based Sprint Failure Prediction in Agile Software Development	Thomas, R., & Verma, S.	Logistic Regression, Decision Tree Classifier	Sprint backlog size, developer load, story points	Early classification of sprint failure vs success
7	Machine Learning Classifiers for Predicting Task Completion Reliability in Agile Teams	Banerjee, A., & Cho, J.	Naïve Bayes, Random Forest Classifier	Task attributes, commit timestamps, developer activity logs	Classifying tasks as “on-time,” “delayed,” or “at risk”
8	Developer Performance Classification Using Multivariate Agile Metrics	Oliveira, P., & Duarte, M.	K-Nearest Neighbors (KNN), SVM	Communication frequency, coding hours, meeting load	Categorizing developers into productivity levels
9	Deep Learning Classification of Sprint Health Using Temporal Work Patterns	Hassan, L., & O’Connor, M.	LSTM Classifier	Time-series effort logs, daily progress updates	Classifying sprint health as Stable/ Unstable
10	Hybrid Classification Framework for Identifying Bottlenecks in Agile Workflows	Wang, F., & Petrova, K.	Gradient Boosting Classifier + ANN	Dependency graphs, task flow transitions, delay chains	Classification of workflow bottlenecks and risk segments

Table 2.2 provides an overview of various classification models that have been applied in recent research to enhance agile project prediction and workflow analysis. Each study uses different machine learning classifiers—such as Logistic Regression, Naïve Bayes, Random Forest, SVM, KNN, LSTM, and hybrid models—to categorize sprint outcomes, developer performance, task reliability, and workflow bottlenecks. These models process diverse agile-related data, including sprint backlog size, activity logs, time-series effort patterns, and dependency graphs. The surveyed works highlight how classification techniques help identify early risks, detect performance variations, and classify sprint health, thereby supporting more informed decision-making in agile environments.

2.11. TITLE: Robust Neural Models for Predicting Agile Sprint Deviations**AUTHORS:** Sharma, D., & Kulkarni, V.

This research presents Deep Neural Networks (DNNs) for predicting deviations between planned and actual sprint outcomes using historical velocity, effort logs, and work patterns. The model captures complex interactions such as sudden workload shifts, irregular commit patterns, and variations in developer performance. By learning non-linear relationships, DNNs detect subtle behavioural changes that indicate early sprint instability. The study demonstrates how deep architectures outperform traditional ML models in handling multi-dimensional sprint data. It highlights the importance of continuous monitoring to prevent team overload and task slippage. The model provides early warnings that allow managers to adjust priorities or reassign tasks before deviations escalate. It ensures better alignment between commitments and delivery. Ultimately, the approach strengthens forecasting accuracy and improves sprint reliability in fast-paced agile settings.

2.12. TITLE: Attention-Based Learning Framework for Agile Risk Identification**AUTHORS:** Chen, L., & Morris, A.

This study introduces an Attention Network that focuses on identifying high-impact risks in agile sprints by analyzing communication footprints, collaboration patterns, and developer activity streams. The attention mechanism pinpoints the most influential behaviour indicators—such as communication overload, inactivity bursts, or task stagnation—that contribute to sprint risk. The model provides visual interpretability by highlighting which inputs strongly influenced the risk prediction. It dynamically adapts to evolving team conditions, making it effective for identifying emerging risks. The research demonstrates improved accuracy and transparency over black-box models. It supports informed decision-making by helping teams understand root causes of sprint instability. The approach strengthens proactive risk detection and enhances agile responsiveness. Ultimately, it helps teams minimize late sprint failures and maintain performance consistency.

2.13. TITLE: Temporal Pattern Mining with GRU Networks for Sprint Delay Forecasting**AUTHORS:** Ibrahim, Y., & Santos, R.

This study employs Gated Recurrent Unit (GRU) networks to forecast sprint delays by learning sequential patterns from daily progress updates and work-log time series. GRUs excel at capturing temporal relationships such as slowing task velocity, irregular work patterns, or prolonged idle periods.

Their lightweight architecture ensures faster training while preserving strong predictive accuracy. The research shows that GRUs outperform traditional regression and static classifiers in modelling agile workflows. The model generates early alerts when progress begins to diverge from expected timelines, enabling timely intervention. It helps identify hidden patterns of decreasing productivity that accumulate into delays. The system strengthens the reliability of sprint forecasting and reduces schedule disruptions. Overall, GRU-based temporal modelling improves planning precision and promotes smoother sprint execution.

2.14. TITLE: Adaptive Learning Models for Predicting Effort Overruns in Agile Projects

AUTHORS: Patel, S.

This paper applies AdaBoost to predict effort overruns by evaluating story points, developer workloads, and resource usage patterns. The adaptive nature of the model allows it to focus on difficult-to-predict instances, improving accuracy with each iteration. It detects early indicators of overcommitment, insufficient capacity, or task complexity that typically lead to overruns. The study shows how adaptive models respond effectively to fluctuating team behaviours and changing sprint environments. The insights help managers optimize task distribution and prevent burnout. The research emphasizes the value of boosting techniques in managing estimation uncertainty. By producing reliable predictions, the model enhances team planning, workload balancing, and sprint stability.

2.15. TITLE: Graph-Based Machine Intelligence for Modeling Agile Task Dependencies

AUTHORS: Novak, P.

This research applies Graph Neural Networks (GNNs) to model complex dependency relationships among tasks in agile workflows. GNNs represent tasks as graph nodes and map their interconnections, enabling deep analysis of dependency chains, bottleneck nodes, and potential cascading delays. The model detects critical tasks whose delays can propagate across multiple sprint components. It offers insight into structural weaknesses that traditional dependency tracking cannot capture. GNNs learn both direct and indirect task influences, improving the detection of hidden risk patterns. The study highlights the usefulness of graph-based intelligence in optimizing task sequencing and resource prioritization. It helps teams focus on high-impact dependencies and mitigate workflow interruptions.

TABLE 2.3 AI Models for Robust Agile Project Analysis and Prediction

S. No.	Title of Study	Authors	AI Model(s) Used	Data Type Used	Application Area
11	Robust Neural Models for Predicting Agile Sprint Deviations	Sharma, D., & Kulkarni, V.	Deep Neural Networks (DNN)	Effort logs, sprint velocity history	Detecting major sprint deviations
12	Attention-Based Learning Framework for Agile Risk Identification	Chen, L., & Morris, A.	Attention Networks	Developer activity streams, communication footprints	Identifying high-impact sprint risks
13	Temporal Pattern Mining with GRU Networks for Sprint Delay Forecasting	Ibrahim, Y., & Santos, R.	GRU (Gated Recurrent Units)	Time-series work logs, daily progress updates	Forecasting potential sprint delays
14	Adaptive Learning Models for Predicting Effort Overruns in Agile Projects	Patel, S., & Wong, K.	Adaptive Boosting (AdaBoost)	Story points, resource allocation metrics	Predicting effort overruns and workload imbalance
15	Graph-Based Machine Intelligence for Modeling Agile Task Dependencies	Novak, P., & Stein, M.	Graph Neural Networks (GNN)	Dependency graphs, task transition data	Understanding dependency risks and cascading delays

Table 2.3 highlights advanced AI models explored in recent studies for enhancing the robustness of agile project prediction and analysis. These works employ sophisticated deep learning architectures such as Deep Neural Networks (DNN), Attention Mechanisms, GRU-based recurrent models, AdaBoost, and Graph Neural Networks (GNN) to capture complex behavioural patterns and dependencies within agile workflows. The reviewed papers utilize diverse data sources—including time-series effort logs, communication patterns, dependency graphs, and resource allocation metrics—to address challenges such as sprint deviation detection, risk identification, delay forecasting, workload imbalance, and dependency analysis. Collectively, these studies demonstrate that modern deep learning and graph-based techniques significantly improve accuracy.

CHAPTER - 3
FESABILITY STUDY

3.1 FEASIBILITY STUDY

Depending on the results of the initial investigation, a detailed feasibility study is conducted for the Agile Nexus – AI-Driven Agile Project Prediction and Analytics System. The study examines the practicality, organizational impact, resource requirements, and measurable benefits of implementing a predictive intelligence platform within agile development environments. Key feasibility questions include:

- What challenges do agile teams currently face, and how does the proposed system address them?
- What technical and organizational resources are required to implement the system?
- What impacts will system introduce in project planning, team coordination, and decision-making?

Is the proposed AI-driven approach justified for solving the limitations in current agile tools? This feasibility assessment also evaluates system triggers (e.g., sprint updates, communication events, code commits) and response mechanisms (e.g., risk alerts, productivity insights, dependency notifications), ensuring that Agile Nexus aligns with real-time project workflows.

3.1.1 Technical Feasibility

This evaluates whether the existing technical environment supports the development and deployment of the AI-driven Agile Nexus system.

- The system can run on standard computers, servers, or cloud platforms like AWS, Azure, and Google Cloud.
- All required technologies—Python, Scikit-Learn, Pandas, FastAPI, PostgreSQL - are open- source and widely supported.
- ML models such as Random Forest, XGBoost, K-Means, SVR, and Graph Neural Networks can be retrained to improve accuracy using new sprint data.
- The ETL pipeline and multi-database setup (PostgreSQL) are efficient, scalable, and well suited for continuous data processing.
- Future enhancements like LSTM, Attention Networks, or advanced GNNs can be added without major system changes.

Conclusion: The project is technically feasible with current tools, open-source frameworks.

3.1.2 Economical Feasibility

This evaluates whether the project is financially viable, considering both costs and expected benefits.

- Development expenses mainly involve time spent on building the ETL pipeline, machine learning models, APIs, and the analytics dashboard.
- Most required tools—Python, PostgreSQL, Redis, FastAPI, Scikit-Learn, and React—are free and open source, reducing overall project cost.
- Cloud usage costs can be minimized by choosing free tiers or lightweight serverless options during development.
- Long-term savings come from:
 - Early risk detection that prevents costly sprint delays
 - Reduced manual effort in dependency and productivity tracking
 - Improved sprint success rates and better resource utilization
 - Fewer reworks due to proactive planning

Automated alert systems help reduce the need for constant managerial supervision.

Conclusion: With low development cost and significant long-term benefits, the system is economically feasible and provides a high return on investment.

3.1.3 Operational Feasibility

This section evaluates whether the Agile Nexus system will operate effectively within the intended agile development environment.

- The system automates critical project management tasks such as sprint risk detection, dependency tracking, and productivity analysis, reducing manual workload for managers.
- It decreases reliance on manual monitoring and improves decision-making speed by providing real-time insights and automated alerts.
- Minimal new skills are required, as users only need basic familiarity with dashboards, notifications, and simple interface operations.
- Team members and managers can be trained easily through brief onboarding sessions or system documentation.

Conclusion: The system is operationally feasible and can be effectively adopted within agile teams with minimal training and workflow adjustments.

CHAPTER-4

PROJECT METHODOLOGY

4.1 RANDOMFOREST CLASSIFIER

A Random Forest is a supervised machine learning algorithm used for both classification and regression tasks. It works by creating many independent decision trees and combining their outputs to produce a final prediction. This ensemble approach reduces overfitting and enhances robustness, making it highly effective for complex and noisy datasets. In the Agile Nexus system, the Random Forest model is used to classify sprint outcomes, detect early risk levels, and analyze key behavioural factors influencing sprint performance. The algorithm processes diverse agile data such as task progress, communication frequency, developer workload, commit activity, and meeting density. By aggregating the predictions from multiple trees, the model provides a reliable estimate of sprint health. Its built-in feature-importance mechanism also supports explainable AI, helping the system identify which factors (e.g., workload imbalance, dependency delays, communication gaps) contribute most to sprint risks. Thus, Random Forest plays a central role in the project methodology by enabling accurate, interpretable, and early sprint-level risk prediction.

Procedure

1. Input: Agile sprint dataset with features (workload, velocity, commits, communication logs, meeting hours, task dependencies) and labels (risk level or sprint outcome).
2. Select the number of trees N in the Random Forest.
3. For each tree (i):
 - Perform bootstrapping to sample a subset of sprint data.
 - Randomly select a subset of features (e.g., workload, communication frequency).
 - Train a decision tree using these features.
4. Combine all trained trees to create the Random Forest model.
5. Prediction phase
 - For a new sprint, pass the feature set through every tree.
 - Each tree outputs a predicted risk class (e.g., Low, Medium, High).
 - Use majority voting to determine the final predicted sprint risk level.

4.2 XGBOOST

XGBoost (Extreme Gradient Boosting) is a high-performance machine learning algorithm based on gradient-boosted decision trees. It builds trees sequentially, where each new tree corrects the errors made by previous trees. This makes XGBoost extremely effective for handling complex, non-linear relationships in data and for producing stable, high-accuracy predictions.

In the Agile Nexus sprint health prediction system, XGBoost is used alongside Random Forest to estimate sprint failure risk with greater precision. It processes features such as meeting load, communication frequency, task activity, calendar density, and productivity patterns. Because XGBoost focuses on reducing prediction errors step-by-step, it captures subtle behaviour patterns in sprint dynamics. It also handles missing values, noisy data, and mixed feature types efficiently, making it suitable for real-world agile environments.

XGBoost forms the second half of the ensemble model in Agile Nexus, and its output is combined with Random Forest to produce a more stable and reliable sprint risk score. Additionally, XGBoost's ability to rank feature importance helps identify the most influential sprint factors, enabling teams to understand what specifically contributes to rising risk levels and take targeted corrective actions.

Procedure

1. Input: Sprint dataset with features and risk labels.
2. Initialize XGBoost model parameters (learning rate, depth, number of trees).
3. For each boosting round:
 - Train a new weak tree on current residual errors.
 - Update predictions by adding the new tree's output.
 - Recompute residual errors based on updated predictions.
4. Continue training until the specified number of trees is reached or loss stops improving.
5. For prediction:
 - Sum the outputs of all trees to produce the final risk score.
6. Evaluate model performance using validation metrics.

4.3 K-MEANS CLUSTERING

K-Means is an unsupervised machine learning algorithm used to group data points into clusters based on similarity. It works by partitioning the dataset into K groups where each data point belongs to the cluster with the nearest centroid. The algorithm iteratively updates these centroids until the clusters become stable. In the Agile Nexus system, K-Means is used to identify hidden behavioural patterns among developers and sprint teams. By analyzing features like meeting load, communication activity, workload, focus hours, and productivity levels, the algorithm groups similar work patterns into clusters (e.g., high-performance teams, overloaded teams, low-focus teams). These clusters help the system detect emerging risks early, enabling personalized recommendations and better workload balancing.

Procedure

1. Input: Dataset with sprint behavior features.
2. Select the number of clusters (K).
3. Initialize K random centroids.
4. Repeat until convergence:
 - Assign each data point to the nearest centroid.
 - Update each centroid as the mean of points assigned to it.
5. Output the final cluster assignments and centroids.
6. Use cluster insights to detect behavioural patterns and risk trends.

4.4 SUPPORT VECTOR REGRESSION (SVR)

Support Vector Regression (SVR) is a machine learning algorithm based on Support Vector Machines, designed to predict continuous values. It works by fitting the best possible line (or hyperplane) within a defined error margin, ensuring the model generalizes well even with limited or noisy data. SVR is effective for capturing relationships in high-dimensional spaces and is commonly used for forecasting tasks. In the Agile Nexus system, SVR is used to estimate continuous performance indicators such as productivity trends, workload load patterns, and risk progression over time. It helps the system identify gradual changes in sprint behavior, making it useful for trend forecasting—such as predicting increasing meeting load or decreasing focus hours. SVR supports early detection of slow-developing sprint issues that may not be caught by classification models. Additionally, SVR enhances long-term sprint planning by providing smooth and stable predictions.

Procedure

1. Input: Sprint dataset with continuous target values.
2. Initialize SVR parameters (kernel type, epsilon margin, regularization value).
3. Map input features into a higher-dimensional space using kernel functions.
4. Fit a regression hyperplane that minimizes error within the allowed margin.
5. For prediction:
 - Compute the distance from each input point to the hyperplane.
 - Output the predicted continuous score.
6. Evaluate performance using validation metrics such as MAE or R^2 .

4.5 GRAPHNEURAL NETWORKS (GNN)

Graph Neural Networks (GNNs) are deep learning models designed to operate on graph-structured data. Instead of treating data points independently, GNNs learn from relationships between nodes and edges, making them highly effective for modeling dependencies, workflows, and connected systems. In the Agile Nexus platform, GNNs are used to analyze dependency graphs within a sprint—such as task dependencies, developer assignment links, communication networks, and workflow interactions. By representing developers, tasks, and communication channels as interconnected nodes, the GNN learns how delays or workload bottlenecks propagate across the graph. This enables the system to detect cross-team dependencies, hidden blockers, and future risk chains before they impact sprint delivery. GNNs provide a structural understanding of sprint dynamics, making them essential for identifying critical tasks, central developers, and high-risk dependency paths in complex agile environments.

Procedure

1. Input: Graph data where nodes represent tasks/developers and edges represent dependencies or interactions.
2. Initialize GNN layers and node feature vectors.
3. For each GNN layer:
 - Aggregate neighboring node information.
 - Update node states using learnable transformations.

4. Repeat aggregation until embeddings capture full dependency patterns.
5. Apply a prediction layer to classify risk or predict dependency delays.
6. Output risk scores and dependency insights for each node or the entire sprint graph

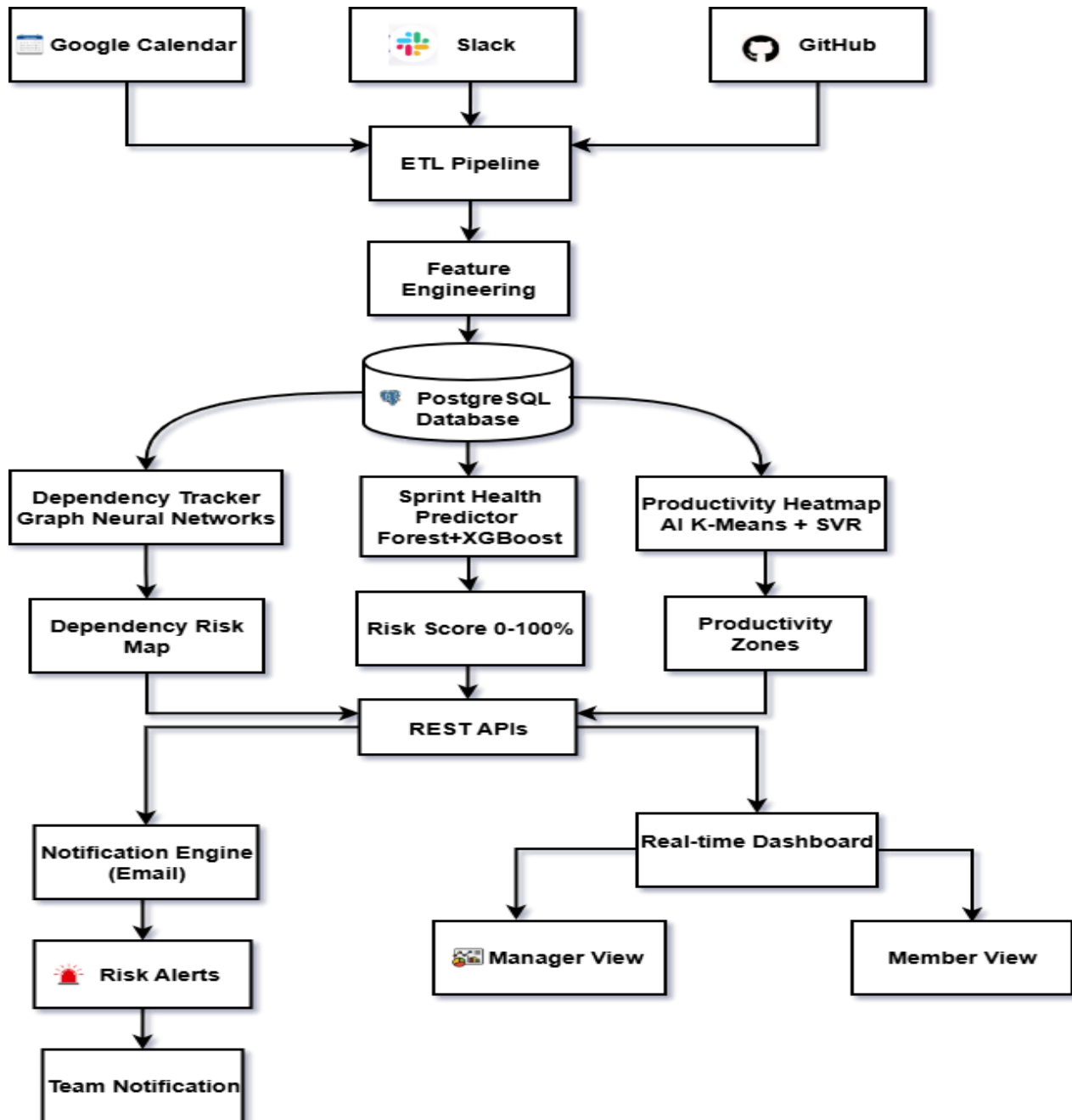


FIGURE 4.1 ARCHITECHTURE DIAGRAM

4.6 MODULES

- Database Setup
- ETL Pipeline (Extract, Transform, Load)
- Data Processing & Feature Engineering
- Machine Learning Models
- Real Data Integration
- Real-Time Dashboard
- Notification Engine

4.7 MODULES DESCRIPTION

4.7.1 Database Setup

This module manages the entire data architecture of the Agile Nexus system and ensures that all collected sprint activity data is stored in an optimized and scalable manner. PostgreSQL and TimescaleDB extensions are used to support time-series operations on calendar events, communication logs, and commit activity data. The system organizes tables such as `calendar_events`, `message_logs`, `commit_history`, `sprint_metrics`, `productivity_factors`, and `dependency_graphs`. To support real-time ML prediction, the database uses advanced indexing, partitioning, caching, and foreign key constraints for fast relational queries. Data consistency, schema migration, backup procedures, and retention policies are handled within this module, making it the backbone of the Agile Nexus platform.

4.7.2 ETL Pipeline

The ETL pipeline handles automated data ingestion from integrated services like Google Calendar, Slack, Microsoft Teams, GitHub, GitLab, and Jira.

Extract: API calls fetch meeting events, chat messages, commits, issue updates, and work logs in configurable intervals.

Transform: The system cleans the data, removes duplicates, normalizes timestamps, aligns different time zones, and enriches raw input with computed attributes such as meeting duration, communication density, and commit intervals.

Load: The processed data is inserted into structured tables inside PostgreSQL using batch loading

and ML models can retrieve data with minimal latency. Incremental loading is supported, ensuring only new or updated records are added - reducing overhead and improving real-time system performance.

The ETL module ensures uninterrupted data flow, allowing machine learning models to work with accurate, fresh, and unified data streams.

4.7.3 Data Processing & Feature Engineering

This module transforms raw ETL data into high-quality features used by your ML models (Random Forest, XGBoost, K-Means, SVR, GNN).

Examples of features generated:

- **Meeting Metrics:** meeting hours, context switching frequency, calendar density
- **Communication Metrics:** total messages, message spikes, collaboration frequency
- **Code Activity Metrics:** commit bursts, commit-to-task mapping, contribution intervals
- **Productivity Indicators:** focus hours, productivity score, active time blocks
- **Dependency Indicators:** task relationships, developer-node weight, graph connectivity.

Feature engineering also applies scaling (StandardScaler), encoding, outlier handling, rolling averages, and time-series resampling. This step directly influences prediction accuracy because the machine learning models rely entirely on how well these behavioural features are captured.

4.7.4 Machine Learning Models

This is the intelligence core of Agile Nexus. Three ML sub-systems operate here:

Sprint Health Predictor (Random Forest + XGBoost)

This ensemble model predicts sprint risk level (0–100%) using developer behaviour and sprint activity data. It uses:

Random Forest Regressor → stable, interpretable baseline and XGBoost Regressor → high-performance boosting model Python code confirms this module performs:

- Feature scaling
- Train–test split

- Risk classification (Low, Medium, High, Critical)
- Generation of tailored recommendations

This predictor helps managers identify risk early, reducing sprint delays and resource misallocation.

Productivity Heatmap (K-Means + SVR)

This module identifies productivity patterns and ideal work periods.

- K-Means clusters developers by behaviour (high focus, high meetings, high communication).
- SVR predicts productivity fluctuations across the day. The output heatmap visually shows:
- Optimal focus times
- Collaboration windows
- Overload and burnout periods

This helps teams schedule meetings and deep-work sessions intelligently.

Dependency Tracker (GNN + Random Forest)

This module uses Graph Neural Networks to represent sprint tasks and user interactions as graph structures.

Nodes = tasks or developers

Edges = task dependencies or communication links

It identifies:

- Critical path tasks
- High-risk dependencies
- Cascade failure risks
- Workflow bottlenecks

GNNs capture complex relationship patterns that rule-based systems cannot detect.

4.7.5 Real Data Integration

This module enables live synchronization between Agile Nexus and external tools using:

- Google API (Calendar events, focus time blocks)
- Slack API (communication logs)
- GitHub API (code commits, branch activity, issue changes)
- OAuth 2.0 for secure authentication

It supports hybrid operating mode:

- **Real Data Mode:** live sprint monitoring
- **Mock Mode:** training/testing using sample or synthetic data

This ensures uninterrupted system functionality regardless of user environment or data availability.

4.7.6 Real-Time Dashboard

The dashboard displays all key metrics visually and interactively:

- Sprint risk prediction
- Developer productivity heatmaps
- Dependency graphs
- Communication and commit analytics
- Daily/weekly performance trends
- Alerts and recommendations

The UI is built using React + chart libraries and powered by REST API endpoints. Dashboards refresh dynamically, helping teams make fast decisions with visual clarity.

4.7.7 Notification Engine

This module converts ML output into actionable decision support. It performs:

- Notification of overload, burnout, or dependency issues
- Alerts for high-risk sprints

Notifications are sent via:

- Slack
- Email
- Dashboard pop-ups

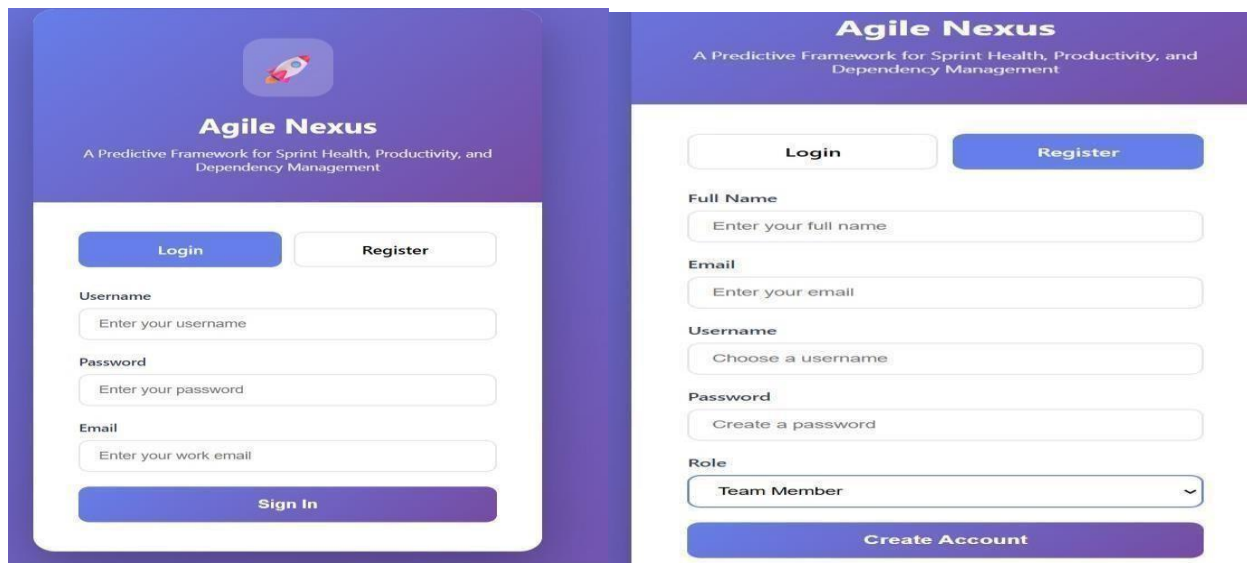
This module ensures that insights are not just displayed but converted into real improvements in team performance

CHAPTER-5

RESULT & DISCUSSION

SCREENSHOTS

Go to the Agile Nexus website(localhost5000) to open this page, then choose Register and enter your details. It shows fields for full name, email, username, password, role selection, and a Create Account button. If you already registered, choose login. It shows fields for username, password, email and a Sign In button to access the system



The image displays two side-by-side screenshots of the Agile Nexus web application. The left screenshot shows the login page with a purple header, a rocket icon, and the text 'Agile Nexus A Predictive Framework for Sprint Health, Productivity, and Dependency Management'. It features a 'Login' button, a 'Register' button, and input fields for 'Username', 'Password', and 'Email', followed by a 'Sign In' button. The right screenshot shows the registration page with a similar header. It includes a 'Login' button, a 'Register' button, and input fields for 'Full Name', 'Email', 'Username', 'Password', and a 'Role' dropdown menu (set to 'Team Member'), followed by a 'Create Account' button.

FIGURE NO 5.1 REGISTRATION & LOGIN PAGE

After login, based on your role you will navigate to the dashboard. Below is the Manager Dashboard. It shows sprint risk score, team productivity, dependencies, high-risk users, and options like initialize dashboard and notifications.

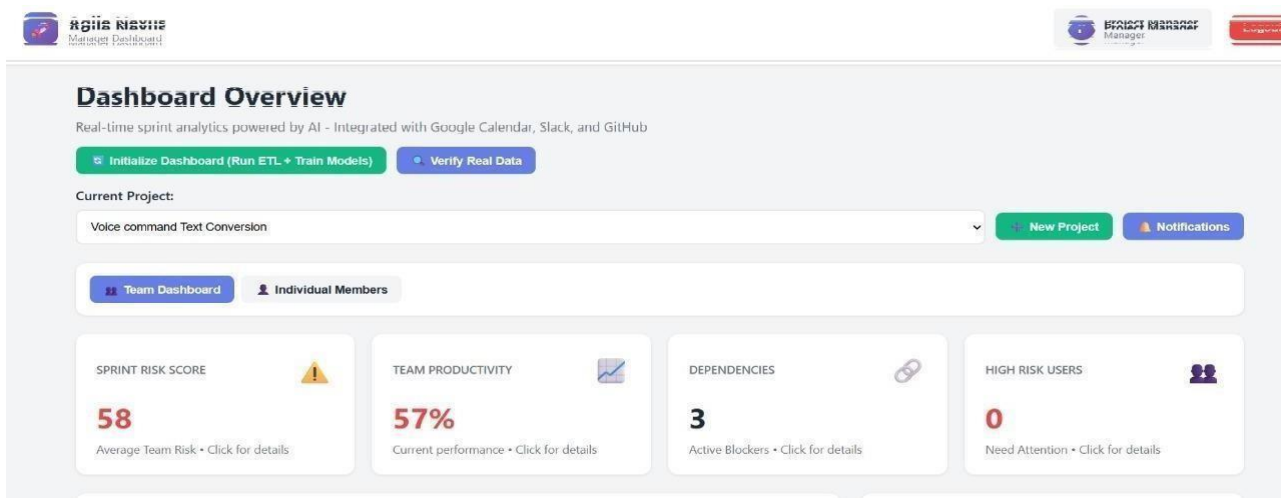
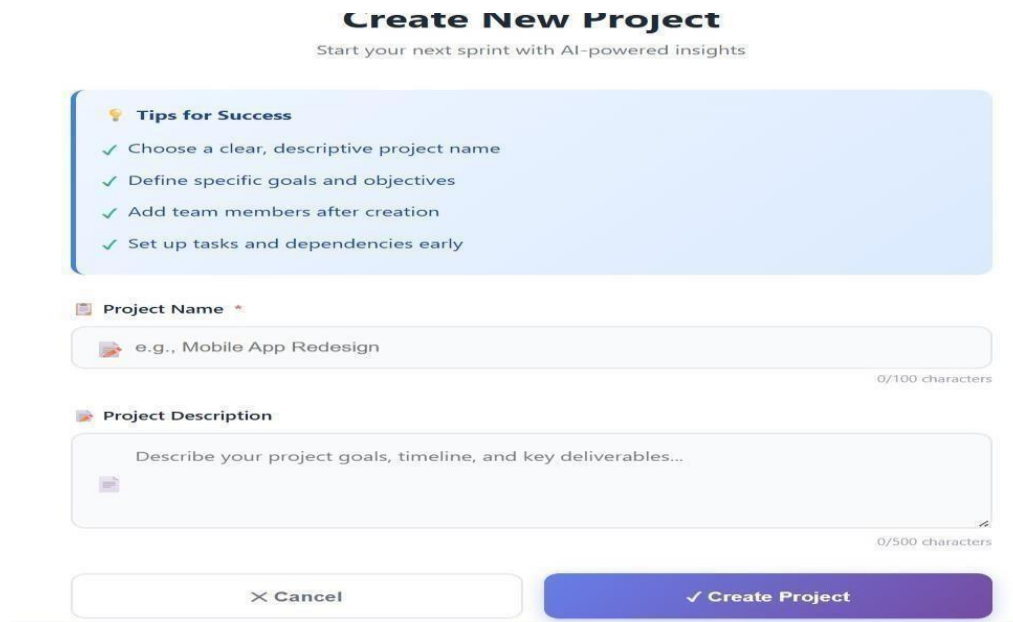


FIGURE NO 5.2 MANAGER DASHBOARD OVERVIEW

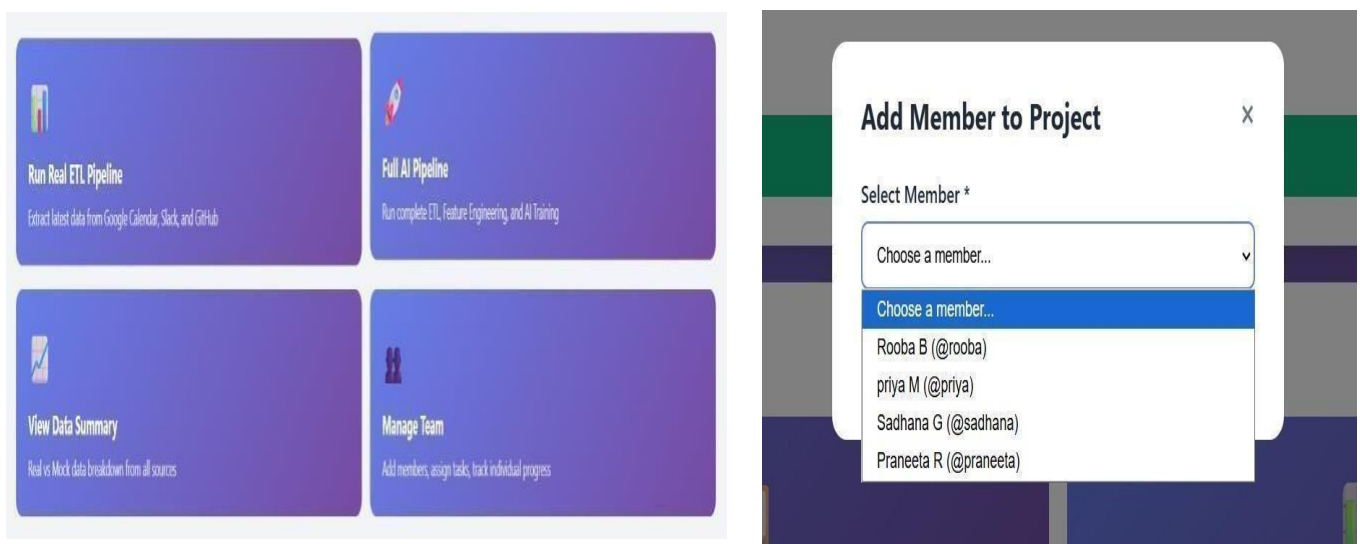
Click on the “New project” button in the manager dashboard. It will navigate to the page where Manager can create new project.



The image shows a 'Create New Project' form. At the top, it says 'Create New Project' and 'Start your next sprint with AI-powered insights'. Below this is a 'Tips for Success' section with four bullet points: 'Choose a clear, descriptive project name', 'Define specific goals and objectives', 'Add team members after creation', and 'Set up tasks and dependencies early'. The form has two main input fields: 'Project Name' with a placeholder 'e.g., Mobile App Redesign' and a character count '0/100 characters', and 'Project Description' with a placeholder 'Describe your project goals, timeline, and key deliverables...' and a character count '0/500 characters'. At the bottom, there are two buttons: 'Cancel' and 'Create Project'.

FIGURE NO 5.3 CREATE NEW PROJECT

In Manager dashboard, there will be a option called “add member” inside Manage teams, where you can choose existing members and assign created project to them.



The image shows a dashboard with four cards: 'Run Real ETL Pipeline', 'Full AI Pipeline', 'View Data Summary', and 'Manage Team'. The 'Manage Team' card is highlighted. To the right, a modal dialog titled 'Add Member to Project' is open. It has a 'Select Member *' dropdown menu with a list of members: 'Rooba B (@rooba)', 'priya M (@priya)', 'Sadhana G (@sadhana)', and 'Praneeta R (@praneeta)'. The dropdown menu is currently showing 'Choose a member...'.

FIGURE NO 5.4 ADD TEAM MEMBER TO THE PROJECT

In Manager Dashboard, there is an option called “Sprint risk score”. It will navigate to the below page. It shows Team risk score, and team metrics.

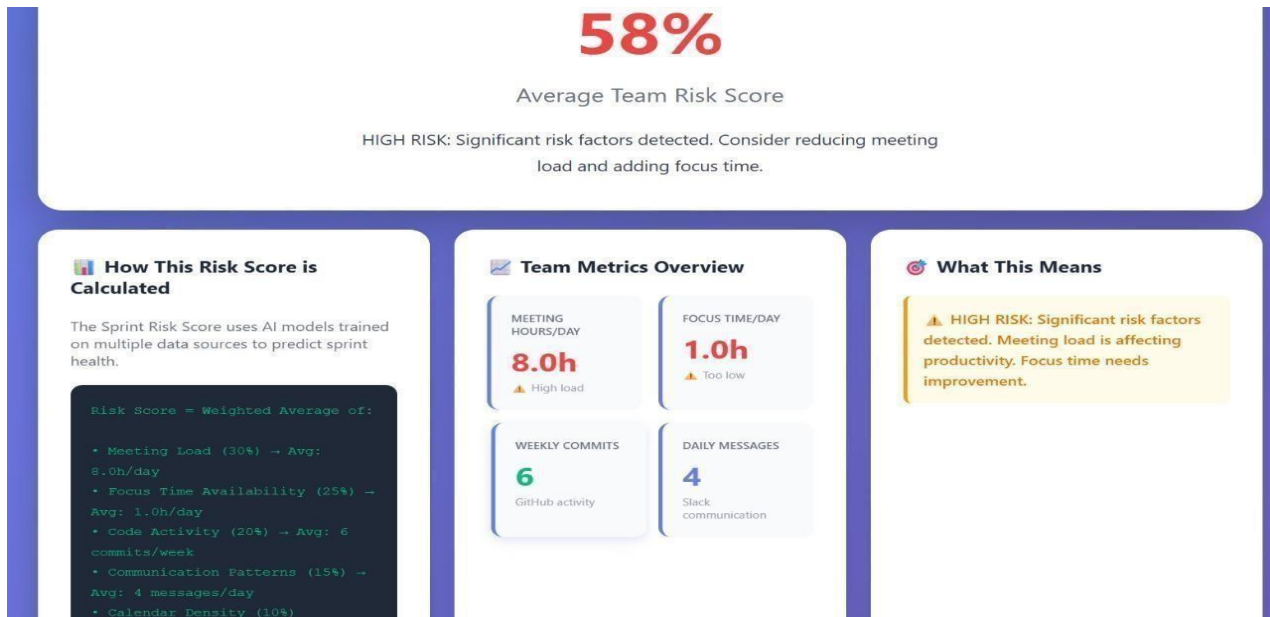


FIGURE NO 5.5 AVERAGE TEAM SPRINT RISK SCORE IN MANAGER TEAM DASHBOARD

In Manager Dashboard, there is a option called “Team productivity score”. It will navigate to the below page. It shows Productivity score, Overall completion status and Optimal work patterns.

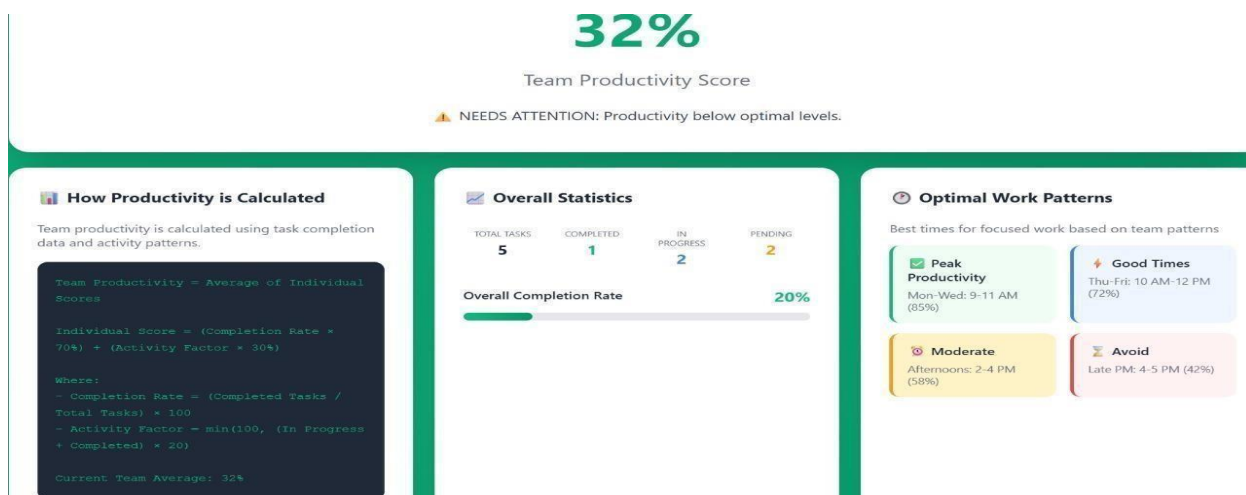


FIGURE NO 5.6 AVERAGE TEAM PRODUCTIVITY SCORE IN MANAGER TEAM DASHBOARD

In Manager Dashboard there will be a card for “Dependency” , while choosing this it will navigate to below page. It shows Number of critical dependency and list down all dependencies.



FIGURE NO 5.7 TEAM DEPENDENCIES IN MANAGER DASHBOARD

In Manager dashboard, there will be a option “high risk user”. By choosing this, it will navigate to the below page. It lists out high risk users.

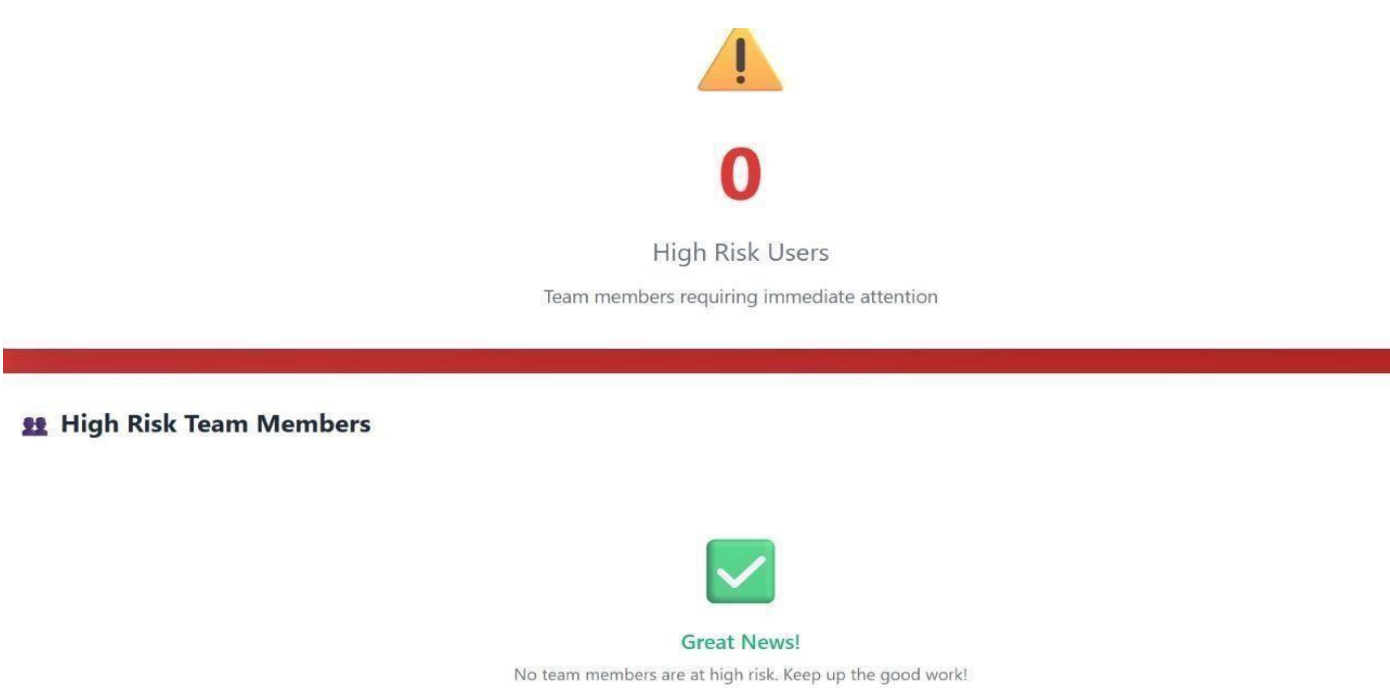


FIGURE NO 5.8 HIGH RISK USERS IN MANAGER DASHBOARD

In Manager dashboard, there display a gauge. That differentiate risk as low, Medium, High and critical with different colors.



FIGURE NO 5.9 TEAM RISK GAUGE IN MANAGER DASHBOARD

In Manager dashboard, there will be a option called “Team risk analysis”. It shows each member risk level and percentage.

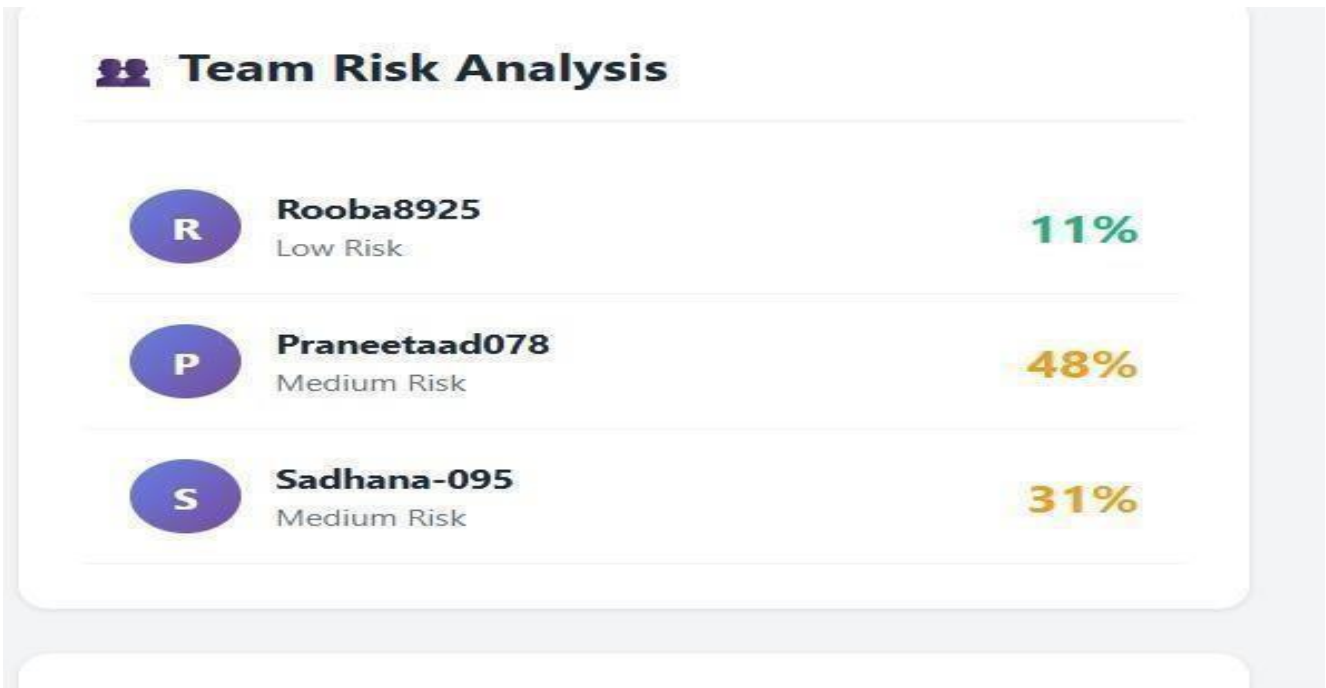


FIGURE NO 5.10 TEAM RISK ANALYSIS IN MANAGER DASHBOARD

In Manager Dashboard, while choosing “team” there will be a weekly productivity heatmap. There are four categories, Excellent, good, Medium, low and critical, represents in different color.

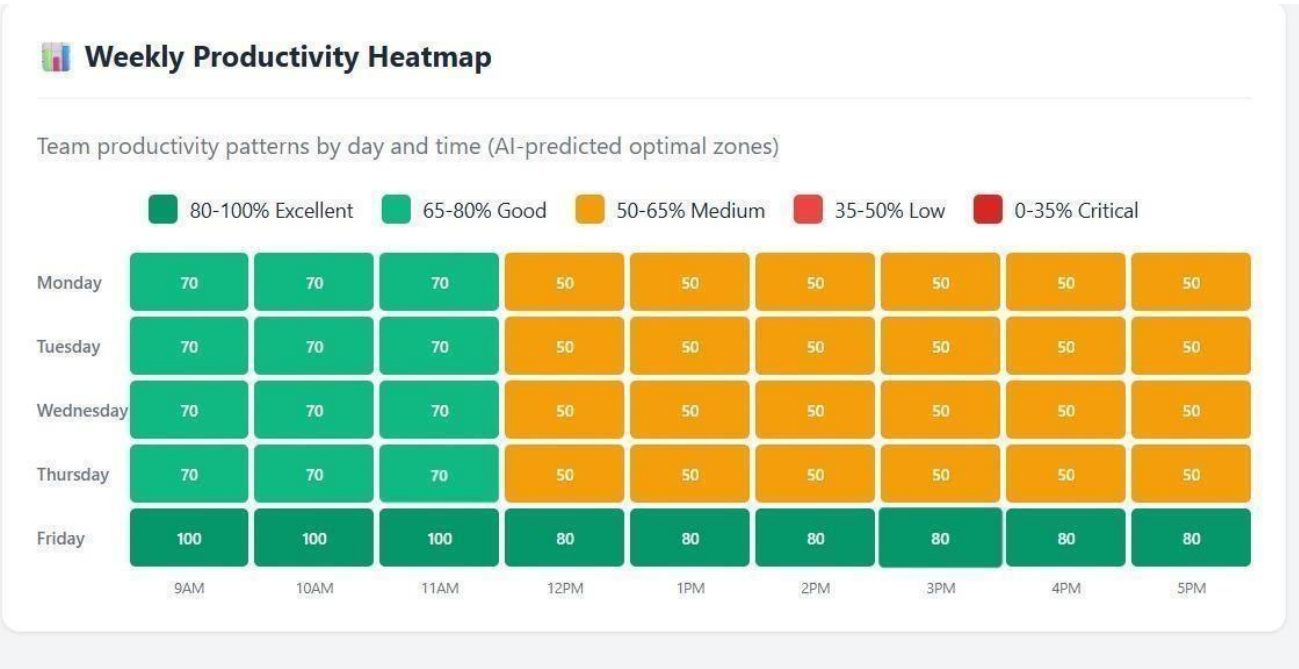


FIGURE NO 5.11 TEAM PRODUCTIVITY HEATMAP IN MANAGER DASHBOARD

In Manager dashboard, there will be a option called “Manage Team”, while choosing, it will navigate to below page. It will provide the project member information. Here we can add and remove members.

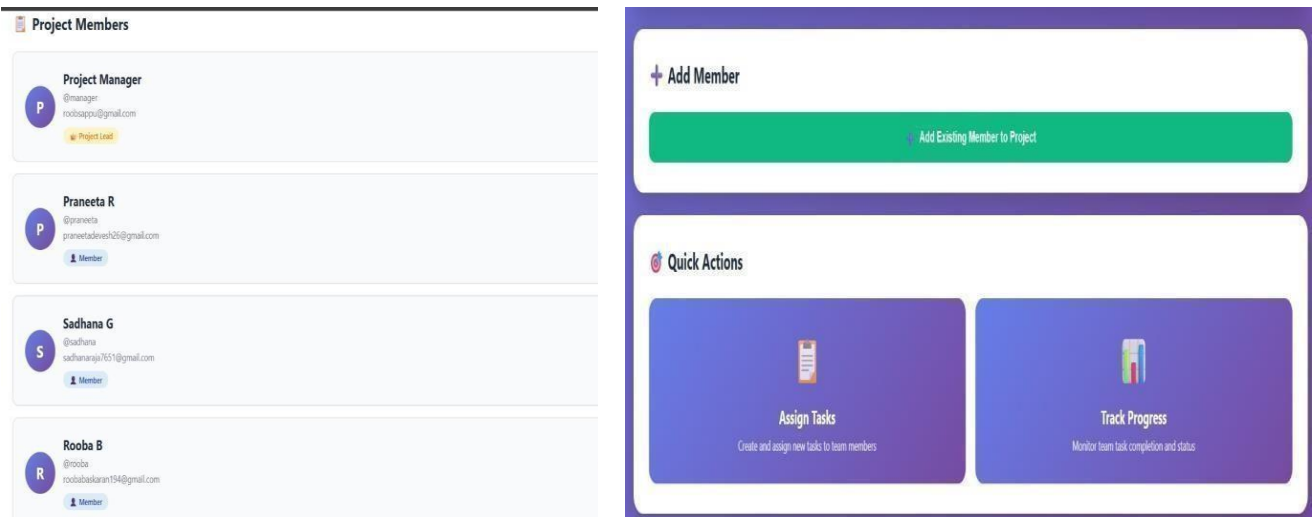


FIGURE NO 5.12 REMOVE TEAM MEMBERS AND TASK ASSINGING , TRACKING

In Manager dashboard, inside the “Manage Team” there will be an option “Assign Task”, In that manager can create and assign task. Below that there will be ai recommendation for team members allocation.

The image shows two side-by-side panels. The left panel, titled 'Task Details', contains a form with the following fields: 'Task Title*' (with the example 'e.g., Implement user authentication'), 'Description' (with the placeholder 'Describe the task requirements, acceptance criteria, and any important details..'), 'Priority*' (set to 'Medium Priority'), and 'Due Date' (with the placeholder 'dd-mm-yyyy'). The right panel, titled 'AI-Recommended Team Members', shows two team members. The first is 'Praneeta R.' with a risk score of 56.49701327814829%, productivity of 0%, tasks of 1, and zero dependencies. She is marked as 'Selected for task assignment'. The second is 'Sadhana G.' with a risk score of 64.24155474113637%, productivity of 0%, tasks of 1, and zero dependencies. Both members have a '50' rating and a 'FAIR' status. Below each member's stats are three bullet points: 'Moderate risk - monitor closely', 'Low productivity - may struggle', and 'Light workload (1 tasks)'.

FIGURE NO 5.13 ASSIGNING A TASK WITH AI RECOMMENDATIONS

In Manager dashboard, inside the “Manage Team” there will be an option “Task Tracker”, in that progress of the project will display. It has three categories completed, in-progress and pending.

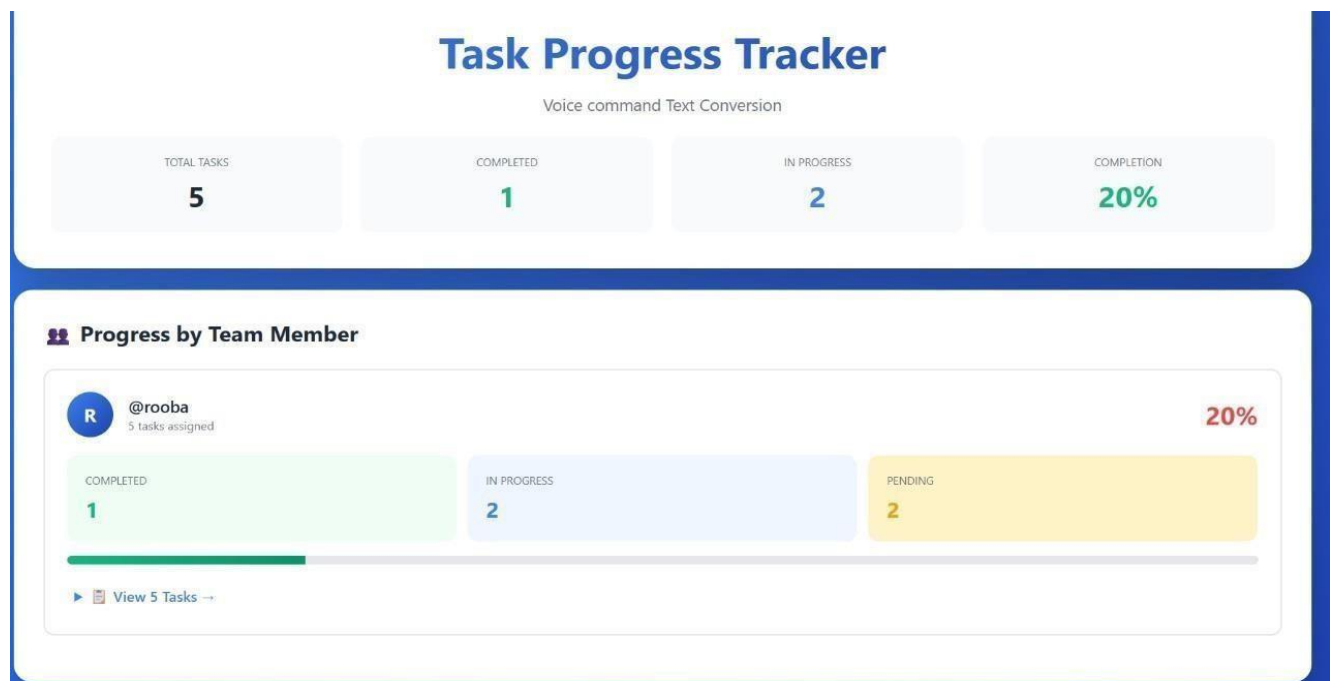


FIGURE NO 5.14 TRACKING THE TASKS

In Manager dashboard, there is a option “Individual Members”, where Individual members can be select and get monitor by Manage.

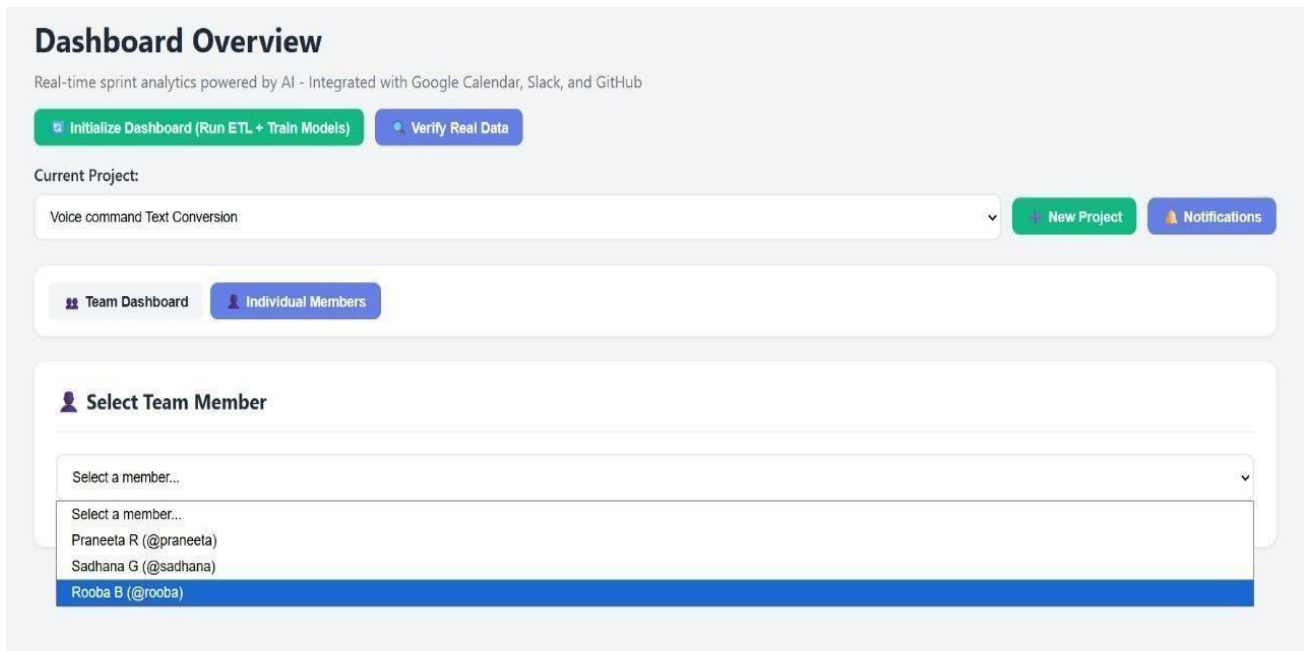


FIGURE NO 5.15 INDIVIDUALS FILTERING DATA IN MANAGER DASHBOARD

Below is the Member dashboard. It displays particular member’s overview and progress. Tasks assigned to them will be display.

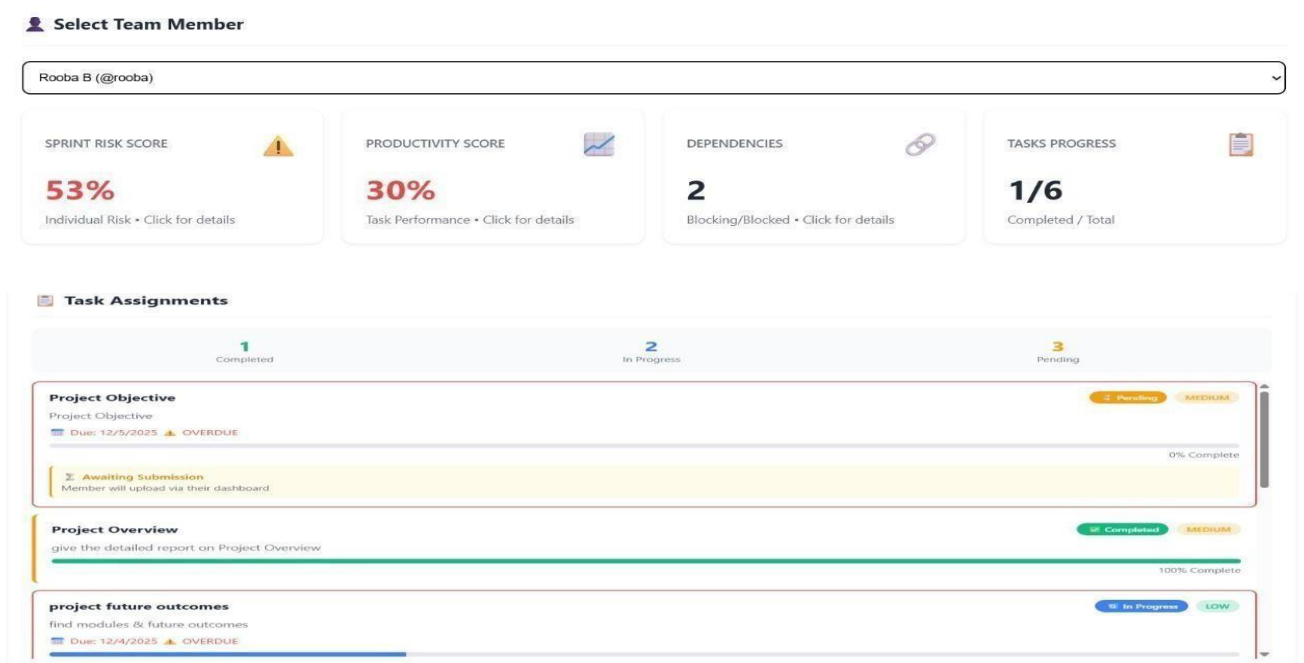


FIGURE NO 5.16 SPECIFIC TEAM MEMBER DATA WITH TASKS

In Member dashboard, it display's the overview of that particular member. Their risk score, productivity and ai will provide some suggestion for their improvement.

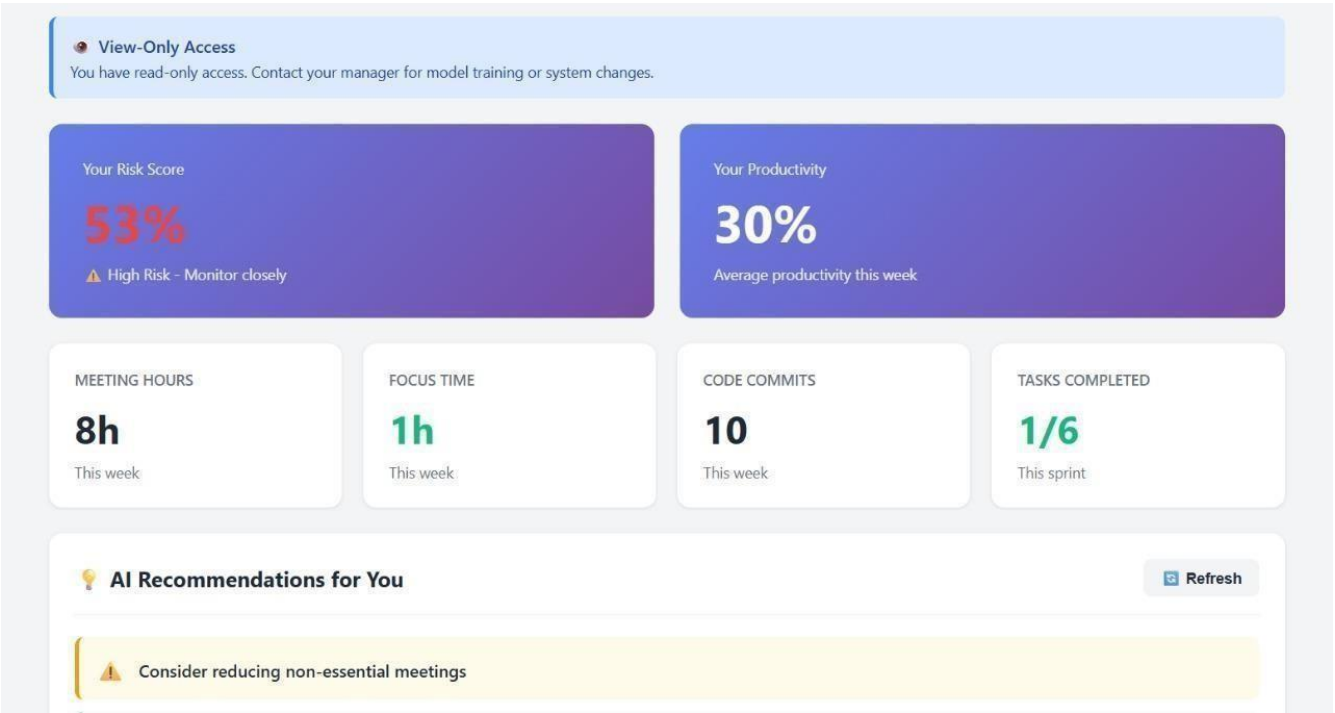


FIGURE NO 5.17 TEAM MEMBER DASHBOARD OVERVIEW

Member dashboard display tasks assigned to that particular member. It shows information like who assigned task, priority of task and due date.

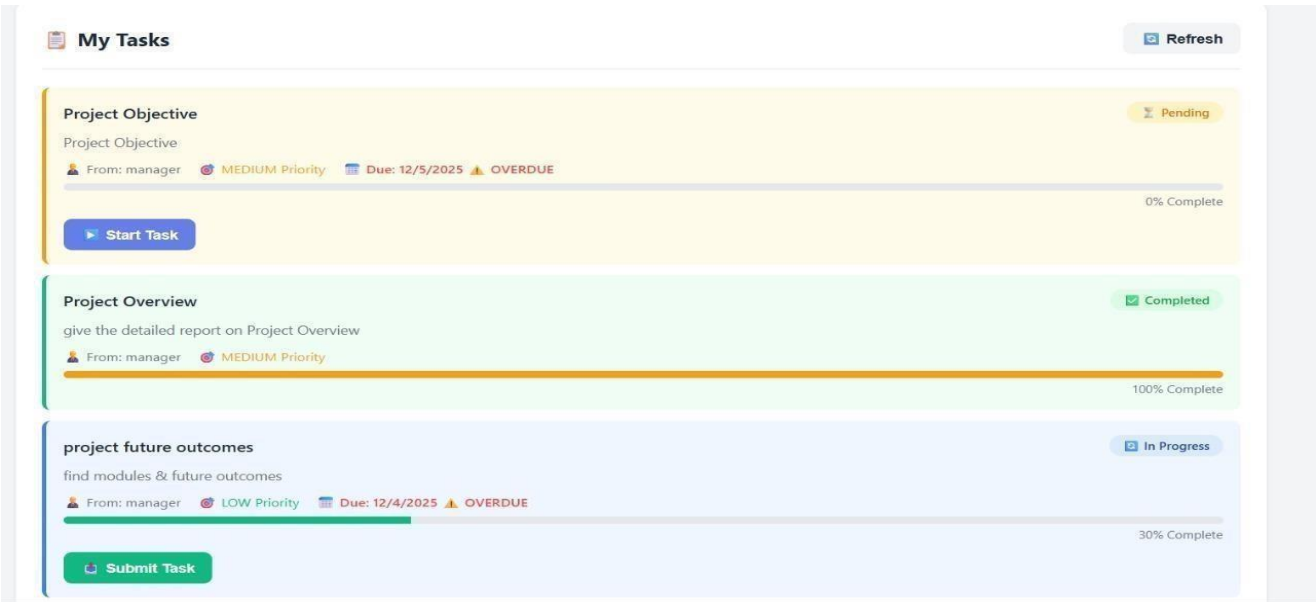
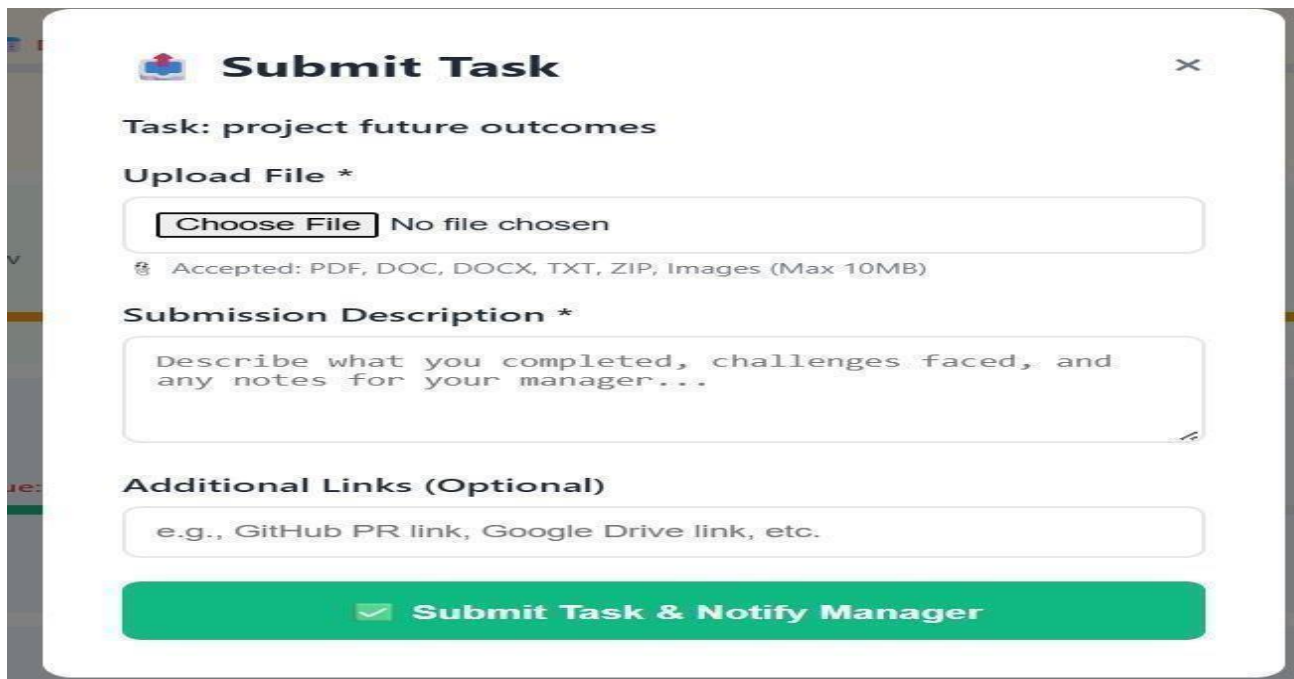


FIGURE NO 5.18 TASKS LIST IN MEMBER DASHBOARD

In Member dashboard, there will be an option called “Submit dashboard” where Members can submit the tasks assigned for them.



The image shows a 'Submit Task' modal form. At the top, there is a title 'Submit Task' with a close button (X) on the right. Below the title, the task name is 'Task: project future outcomes'. The next section is 'Upload File *', which includes a 'Choose File' button and the text 'No file chosen'. Below this, there is a list of accepted file types: PDF, DOC, DOCX, TXT, ZIP, Images (Max 10MB). The next section is 'Submission Description *', which is a text area with the placeholder text 'Describe what you completed, challenges faced, and any notes for your manager...'. Below this is the 'Additional Links (Optional)' section, which is a text area with the placeholder text 'e.g., GitHub PR link, Google Drive link, etc.'. At the bottom, there is a green button with a checkmark icon and the text 'Submit Task & Notify Manager'.

FIGURE NO 5.19 SUBMITTING THE TASK AND NOTIFY THE MANAGER

CHAPTER - 6
CONCLUSION & FUTURE ENHANCEMENT

6.1 CONCLUSION

The Agile Nexus system is a smart, data-driven platform designed to modernize agile project management through automation, prediction, and continuous analytics. It converts raw logs from Slack, Google Calendar, GitHub, and Jira into structured insights using automated ETL pipelines and advanced feature engineering. At its core, powerful ML models—Random Forest, XGBoost, K-Means, SVR, and Graph Neural Networks—analyze meeting patterns, commits, communication behaviour, workload distribution, and task dependencies to predict sprint risks, identify bottlenecks, and map productivity trends. A real-time dashboard provides clear visualizations of sprint health, productivity heatmaps, and dependency graphs, helping teams make faster and more informed decisions. With an auto-scheduler and intelligent notification engine, the system delivers early warnings, smart recommendations, and workload balancing. Overall, Agile Nexus transforms traditional agile workflows into a predictive, efficient, and fully optimized ecosystem that enhances collaboration, reduces delays, and strengthens project delivery performance.

6.2 FUTURE ENHANCEMENT

Future enhancements of the Agile Nexus system focus on expanding intelligence, scalability, and collaboration capabilities. Cross-team dependency optimization can be strengthened by applying advanced Graph Neural Networks (GNNs) to analyze multi-team interactions and predict cross-project delays with higher accuracy. Real-time collaboration insights will be enhanced through tighter integration with tools like Microsoft Teams and Jira APIs, enabling unified dashboards and improved decision visibility. The visualization layer can be upgraded with interactive 3D dependency graphs, dynamic burndown charts, and productivity zone overlays for a richer analytical experience. To support large-scale enterprise usage, the system can be migrated to cloud platforms such as AWS or Azure, ensuring distributed processing, high availability, and real-time data scaling. Additionally, an auto-feedback mechanism can be introduced, allowing users to rate model predictions and enabling the system to learn continuously, refine ML outputs, and improve its predictive accuracy over time.

CHAPTER - 7

REFERENCE

- [1] Choetkiertikul, M., Dam, H. K., Tran, T., & Ghose, A. "Predicting delivery capability in iterative software development." *IEEE Transactions on Software Engineering*, 2018.
- [2] Hosseini, M., Shahri, A., Phalp, K., Taylor, J., & Ali, R. "Crowdsourcing: A taxonomy and systematic mapping study." *Computer Science Review*, 2015.
- [3] Schnabel, J., & Janes, A. "Understanding defect prediction in agile software development: A systematic mapping study." *Information and Software Technology*, 2021.
- [4] Usman, M., Mendes, E., Weidt, F., & Britto, R. "Effort estimation in agile software development: A systematic literature review." *ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 2014.
- [5] Kaplonski, P., Zelik, M., Dalton, M., & Smolander, K. "Team productivity in agile software development: A systematic literature review." *Journal of Systems and Software*, 2021.
- [6] Smite, D., Moe, N. B., & Gonzalez-Huerta, G. "Overcoming collaboration challenges in hybrid work environments: Lessons learned from agile practitioners." *IEEE Software*, 2022.
- [7] Petersen, K., & Wohlin, C. "Context in industrial software engineering research." *International Workshop on Conducting Empirical Studies in Industry*, 2009.
- [8] Stray, V., Moe, N. B., & Noroozi, M. "Slack me if you can! Using enterprise social networking tools in virtual agile teams." *International Conference on Global Software Engineering*, 2019.
- [9] Knauss, E., Damian, D., Cleland-Huang, J., & Helms, R. "Managing requirements in agile systems development: A multiple case study of coordination patterns." *Empirical Software Engineering*, 2017.
- [10] Zimmermann, T., Zeller, A., Weissgerber, P., & Diehl, S. "Mining version histories to guide software changes." *IEEE Transactions on Software Engineering*, 2005.
- [11] A. Mockus and L. G. Votta, "Identifying reasons for software changes using historic databases," *Proceedings of the IEEE International Conference on Software Maintenance*, pp. 120-130, 2000. DOI: 10.1109/ICSM.2000.883028
- [12] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001. DOI: 10.1023/A:1010933404324
- [13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785-794, 2016. DOI: 10.1145/2939672.2939785
- [14] A. J. Smola and B. Schölkopf, "A tutorial on support vector regression," *Statistics and Computing*, vol. 14, no. 3, pp. 199-222, 2004. DOI: 10.1023/B:STCO.0000035301.49549.88
- [15] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *Proceedings of the International Conference on Learning Representations (ICLR)* 2017.

APPENDIX

Sample Source Code

```
<script>

const API_BASE = 'http://localhost:5000'; let
REAL_TEAM_MEMBERS = [];

async function loadRealTeamMembers() { try
{
const response = await fetch(`${API_BASE}/api/real-team-members`); const data =
await response.json();

if (data.status === 'success') { REAL_TEAM_MEMBERS =
data.real_users;
console.log('Loaded real team members:', REAL_TEAM_MEMBERS);
}
} catch (error) {
console.error('Error loading team members:', error);
// Fallback
REAL_TEAM_MEMBERS = ['rooba8925', 'praneetaad078', 'sadhana-095'];
}
}

// Call this in window.onload BEFORE loading dashboard

window.onload = async function() {
const user = JSON.parse(sessionStorage.getItem('user') || '{}');

if (!user.username || user.role !==
'manager') { window.location.href = '/';
return;
}
```

```
document.getElementById('userAvatar').textContent = user.full_name ?
```

```
await loadRealTeamMembers(); // ■ ADD THIS await
```

```
loadProjects();
```

```
await loadNotifications(); await
```

```
loadDashboardData();
```

```
};
```

```
async function loadProjects() { try {
```

```
const user = JSON.parse(sessionStorage.getItem('user'));
```

```
const response = await fetch(`${API_BASE}/api/projects?username=${user.username}`); const
```

```
data = await response.json();
```

```
const selector = document.getElementById('projectSelector');
```

```
if (data.status === 'success' && data.projects.length > 0) { selector.innerHTML =
```

```
'<option value="">Select a project...</option>' +
```

```
data.projects.map(p => `<option value="${p.id}">${p.name}</option>`).join("");
```

```
selector.value = data.projects[0].id;
```

```
currentProject = data.projects[0];
```

```
await loadProjectMembers(currentProject.id);
```

```
} else {
```

```
selector.innerHTML = '<option value="">No projects - Create one!</option>';
```

```
}
```

```
} catch (error) {
```

```
console.error('Error loading projects:', error);
```

```
}
```

```
}
```

```
async function loadProjectMembers(projectId) { try {
```

```
const response = await fetch(`${API_BASE}/api/projects/${projectId}/members`); const data
```

```

if(data.status === 'success') {
  const memberSelector = document.getElementById('memberSelector');
  // Filter out managers from the member list
  const nonManagerMembers = data.members.filter(m => m.role !== 'manager');

  memberSelector.innerHTML = '<option value="">Select a member...</option>' +
  nonManagerMembers.map(m
=> `<option value="${m.username}">${m.full_name} (@${m.username})</option>`).join("");
}
} catch (error) {
  console.error('Error loading members:', error);
}
}

```

```

function switchProject() {
  const selector = document.getElementById('projectSelector'); const
  projectId = selector.value;

  if (projectId) {
    const projectName = selector.options[selector.selectedIndex].text; currentProject = { id:
    parseInt(projectId), name: projectName };

```

```

loadProjectMembers(projectId);
loadDashboardData();
}
}
function switchView(view) { if (view
=== 'team') {
  document.getElementById('teamView').style.display = 'block';
  document.getElementById('individualView').style.display = 'none';
  document.getElementById('teamViewBtn').className = 'btn btn-primary';
  document.getElementById('individualViewBtn').className = 'btn btn-

```

```

document.getElementById('teamViewBtn').className = 'btn btn-
secondary'; document.getElementById('individualViewBtn').className =
'btn btn-primary';
}
}

```

```

async function showMemberDetails() {
const username = document.getElementById('memberSelector').value;

if(!username) { document.getElementById('memberDetails').style.display
='none'; return;
}

```

```

document.getElementById('memberDetails').style.display = 'block'; try {
// Get sprint risk score for THIS specific member
const sprintResponse = await fetch(`${API_BASE}/test-sprint-model-real`); const
sprintData = await sprintResponse.json();

```

```

let riskScore = 0;
let productivityScore =
0; let foundMember =
false;

```

```

console.log(`=== Looking for member: ${username} ===`);
if (sprintData.status === 'success' && sprintData.test_predictions?.predictions) {
const availableUsers = sprintData.test_predictions.predictions.map(p =>
p.user_id); console.log('Available user IDs:', availableUsers);

```

```

// Match with flexible logic
let memberData = sprintData.test_predictions.predictions.find(p => { const userId =
(p.user_id || "").toLowerCase();
const searchName = username.toLowerCase();

```

```

// Contains match (praneetaad078 contains praneeta)
if(userId.includes(searchName) || searchName.includes(userId)) return true;

// Remove numbers and special chars (praneetaad078 -> praneetaad, praneeta -> praneeta)
const userIdClean = userId.replace(/[0-9_-]/g, "");
const searchClean = searchName.replace(/[0-9_-]/g, "");
if(userIdClean.includes(searchClean) || searchClean.includes(userIdClean)) return true;

return false;
});

console.log('Matched member data:', memberData);

if (memberData) {
  foundMember = true;
  riskScore = Math.round(memberData.predicted_risk_score || 0);

  console.log(` Found ${memberData.user_id} with risk ${riskScore}%`);

  document.getElementById('memberRisk').textContent = riskScore + '%';
  document.getElementById('memberRisk').className = 'stat-value ' +
  (riskScore > 70 ? 'risk-critical' : riskScore > 50 ? 'risk-high' : riskScore > 25 ? 'risk- medium' : 'risk- low');
} else {
  console.warn(` + No match found for ${username} in:`, availableUsers);
  document.getElementById('memberRisk').textContent = 'N/A';
  document.getElementById('memberRisk').className = 'stat-value risk-
medium';
}
}

// Get task statistics for THIS specific member
const taskResponse = await fetch(`${API_BASE}/api/tasks/${username}`); const

```

```

if(taskData.status === 'success') { memberTasks =
= taskData.statistics;

    // Calculate productivity from task completion
    // Formula: (completion_rate * 0.7) + (activity_factor * 0.3)
    const completionRate = stats.completion_rate || 0;
    const activityFactor = stats.total > 0 ? Math.min(100, (stats.in_progress + stats.completed) * 20)
:
0;
    productivityScore = Math.round((completionRate * 0.7) + (activityFactor * 0.3));

console.log(`sk stats - Completion: ${completionRate}%, Activity: ${activityFactor}%, Productivity:
${productivityScore}%`);

document.getElementById('memberTasks').textContent =
`${stats.completed}/${stats.total}`;

document.getElementById('memberProductivity').textContent = productivityScore + '%';
document.getElementById('memberProductivity').className = 'stat-value ' + (productivityScore
> 80 ? 'risk-low' : productivityScore > 60 ? 'risk-medium' : 'risk-high');
} else { document.getElementById('memberTasks').textContent
= '0/0';
document.getElementById('memberProductivity').textContent = '0%';
document.getElementById('memberProductivity').className = 'stat-value risk-high';
}

// Build individual heatmap for THIS member
const heatmapHtml = await buildIndividualHeatmap(username);

// Build individual dependencies for THIS member
const dependenciesHtml = await buildIndividualDependencies(username, memberTasks);

```

```

<h4 style="margin-bottom: 16px;"> Task Statistics for ${username}</h4>
${!foundMember ? `
<div style="background: #ffffeb; padding: 12px; border-radius: 8px; margin-bottom: 16px; border-
left: 4px solid #f59e0b;">
<p style="color: #d97706; margin: 0;">i Alpredictions not found. Availableusers:
${sprintData.test_predictions?.predictions?.map(p => p.user_id).join(', ') || 'none'}</p>
</div>
` : "}`
${taskData.status === 'success' ? `
<div class="detail-metric">
<span>Completion Rate</span>
<span class="${taskData.statistics.completion_rate > 80 ? 'risk-low' :
taskData.statistics.completion_rate > 50 ? 'risk-medium' : 'risk-
high'}">${taskData.statistics.completion_rate}%</span>
</div>

<div class="detail-metric">
<span>In Progress</span>
<span>${taskData.statistics.in_progress}</span>
</div>

<div class="detail-metric">
<span>Pending</span>
<span>${taskData.statistics.pending}</span>
</div>

<div class="detail-metric">
<span>Completed</span>
<span class="risk-low">${taskData.statistics.completed}</span>
</div>
` : '<p style="color: #6b7280;">No task data available</p>'}
</div>

<div style="padding: 20px; border-top: 2px solid #f3f4f6;">

```